OpenLearnV2 插件开发完全指南

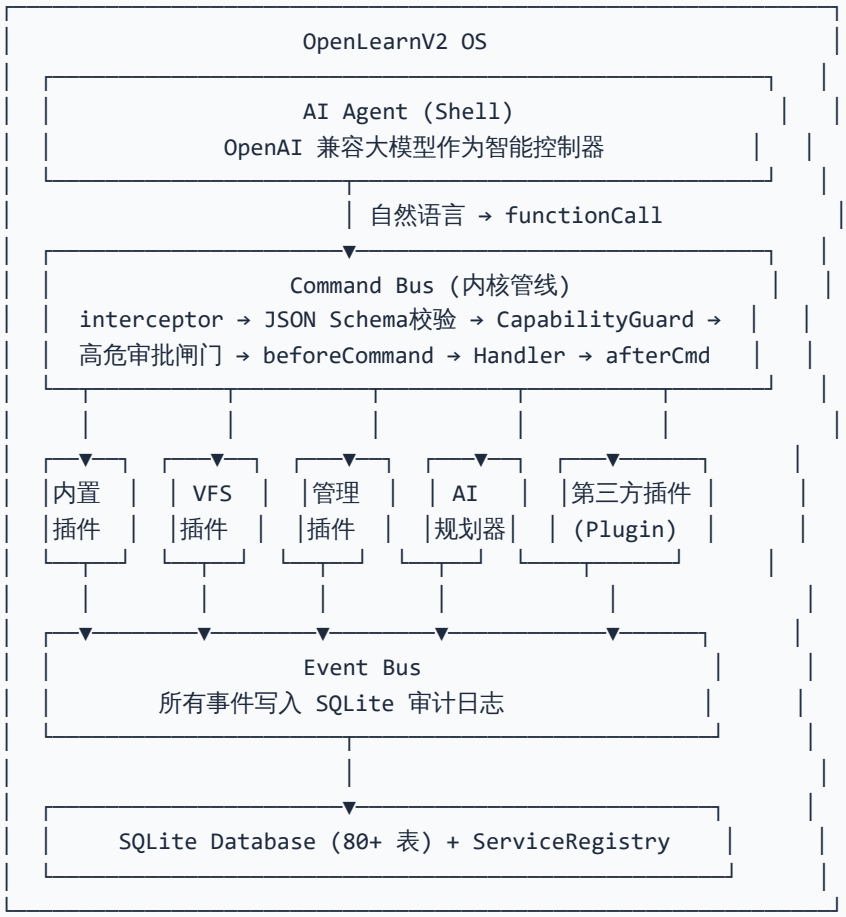
📅 **最后更新**：2026-10-27（§8.1.1 改为 Inline 与隔离模式对比，覆盖 Worker Thread 与子进程两种隔离运行时）

本文档描述 Worker Thread / 子进程隔离模式与 Inline 模式的 API 差异。第三方 ZIP 插件会被强制落到 `process` 模式，请务必阅读 §8.1.1 Inline 与隔离模式 API 差异。

1. 系统架构概述

1.1 设计理念

OpenLearnV2 采用 **插件驱动的命令-事件总线架构**（Plugin-Driven Command-Event Bus）。灵感来源于操作系统内核设计：一个精简的核心内核提供基础能力，所有业务功能通过插件实现。



1.2 核心子系统

子系统	文件	职责
Kernel	`packages/core/kernel/index.ts`	全局单例容器，分层组装所有子系统（Layer 0-3），引导系统插件后
CommandBus	`packages/core/command-bus/index.ts`	命令执行管线：注册 handler → 拦截器链（JSON Schema校验→Cap
EventBus	`packages/core/event-bus/index.ts`	发布/订阅事件，支持通配符 `*`，异步并行通知
ActionRegistry	`packages/core/registry/index.ts`	注册 AI Agent 可发现的工具
CapabilityGuard	`packages/core/capability-system/index.ts`	基于字符串的 RBAC 权限控制
ProcessManager	`packages/core/process-manager/index.ts`	后台进程和定时任务管理
PluginHost	`packages/core/plugin-host/index.ts`	插件生命周期：安装/激活/停用/卸载/热重载，中间件管道
ServiceRegistry	`packages/core/di/service-registry.ts`	依赖注入容器，Token 驱动，依赖图验证
ResourceTracker	`packages/core/plugin-host/resource-tracker.ts`	按 pluginId 管理 Disposable 资源，保证精确清理
WorkerManager	`packages/core/worker-runtime/worker-manager.ts`	Worker Thread 隔离模式管理（`ACTIVATE_TIMEOUT_MS` 常量、dbApi 构
FrontendPluginHost	`src/plugin-host/plugin-host.ts`	前端插件生命周期管理，支持 inline/worker 模式

1.3 数据流

```
用户发送消息 → POST /api/agent/chat
  → AI 模型返回 functionCall（如 lesson.create）
  → executeAgentToolCall() 通过 ActionRegistry.getActionByToolName() 查找 action
  → CommandBus.execute() 执行拦截器管线:
    ├─ JSON Schema payload 校验（基于 action.inputSchema）
    ├─ CapabilityGuard 权限检查（非 admin actor）
    └─ 高危操作 → 写入 pending_commands 审批表 + 抛出异常中断
  → beforeCommand 中间件 → Handler 执行业务逻辑 → afterCommand 中间件
  → EventBus.publish() 发布事件（异步并行通知所有订阅者）
  → Socket.IO 推送给在线客户端
  → 返回结果给 AI Agent（继续对话或结束）
```

2. 开发原理

2.1 插件即 ESM 模块

插件是一个导出了 `manifest` 和 `activate` 函数的 JavaScript/TypeScript 模块：

```
// 插件的最小结构
export default {
  manifest: { ... },
  activate: async (ctx: PluginContext) => { ... },
  deactivate: async () => { ... },  // 可选
};
```

2.2 三档执行模式 —— 先确认你落在哪一档

服务端有**三档**执行模式，第三方插件**默认且强制**落在 `process` ：

模式	谁能用	隔离强度	自定义
`inline`	入口位于 `packages/plugins` 的内置插件，或管理员写进 `OPENLEARN_TRUSTED_INLINE_PLUGINS`	无（与宿主同进程同上下文）	✅ 可
`worker_threads`	同上两类插件	线程级（共享进程，非安全沙箱）	✅ 可
`process`（**第三方默认**）	ZIP 安装、社区注册表安装的一切插件	进程级 + 出站网络默认拒绝	❌ 不

`execution-policy.ts` 的强制改写规则：manifest id 不以 `@openlearn/` 开头、且不在白名单、且文件不在内置目录的插件，安装/更新/激活/启动恢复时**一律被改写为** `process` ，响应里带 `executionModeCoerced: true` 。**manifest id 前缀本身不构成任何信任**。

```
服务端
  PluginHost
    inline   —— 内置插件，同进程直接调用
    worker   —— 受信插件，Worker Thread（共享进程，非安全沙箱）
    process  —— 第三方插件，子进程 + Node 权限 + 出网默认拒绝   ← ZIP 插件在这一层

前端
  FrontendPluginHost
    inline/受信  —— 动态 import()，可注册 React 组件
    process/三方 —— 不透明源隐藏 iframe，只能发数据，不渲染 UI
```

> ⚠️ **这条决定后面一半文档是否适用于你。** 如果你在开发第三方插件（ZIP 分发），你**不能**：
>
> - 提供自定义 React 组件（`component` 是函数，跨 iframe 的 `postMessage` 会抛 `DataCloneError` ）；
> - 渲染任何可见 UI（沙箱 iframe 是 `display: none` 的隐藏执行体）；
> - `import React` 或用 `window.HostSharedDeps` 。
>
> 你能做的是：**manifest 声明式贡献 + 服务端命令**。前端插件指南 与 课件与教学环节扩展 里凡标「仅 inline / worker」的 API，第三方插件都用不了。第三方前端的真实边界见 frontend-sandbox-capability-boundary。

2.3 依赖注入

插件通过 **Token** 声明依赖，由 ServiceRegistry 自动解析注入：

```
import { ICommandBusServiceToken, IDatabaseToken } from '@openlearn/plugin-sdk';

// 在 activate 中解析依赖
const commandBus = await ctx.resolve(ICommandBusServiceToken);
const db = await ctx.resolve(IDatabaseToken);

// 也可以通过 ctx.services 直接访问 9 个内核服务
const eventBus = ctx.services.eventBus;
const ai = ctx.services.ai;
```

V3.2 新增：插件可通过 `ctx.provide()` 向 DI 容器注册自定义服务给其他插件消费：

```
// 插件 A：注册自定义服务
// V3.2: use Token instead of string
await ctx.provide(QuestionBankToken, myQuestionBank);

// 插件 B：消费该服务
const qb = await ctx.resolve({ name: '@my-scope/IQuestionBankService' } as any);
```

2.4 PluginContext 完整接口

```
interface PluginContext {
  // 9 个内核服务接口（直接访问）。注意运行时契约是 `T | null`：
  // 依赖降级或隔离模式未注入时，对应 key 为 null，消费前必须判空。
  services: {
    commandBus: ICommandBusService | null; // 命令执行、注册
    eventBus: IEventBusService | null; // 事件发布/订阅
    actionRegistry: IActionRegistryService | null; // AI 工具注册
    capability: ICapabilityService | null; // 权限管理
    processManager: IProcessService | null; // 后台进程
    storage: IStorageService | null; // K-V 存储
    ai: IAIService | null; // AI 文本生成
    pointsDimension: IPointsDimensionRegistry | null; // 积分维度注册表
    pointsLedger: IPointsLedgerService | null; // 积分账本
    roster: IClassRosterService | null; // 受管名册读取（需 management:read）
  };
  pluginId: string; // 插件 ID
  manifest: Manifest; // 插件 manifest

  // 依赖注入
  resolve<T>(token: Token<T>): Promise<T>; // 从 ServiceRegistry 解析服务
  provide<T>(token: Token<T>, instance: T): Promise<void>; // V3.2: 注册自定义服务

  // 插件专用数据库
  db: PluginDatabaseAPI; // 命名空间隔离的 SQLite 操作（含 migrate() 迁移）

  // V2.5 结构化日志（自动注入 pluginId 和 timestamp）
  log: IPluginLogger; // 支持 debug/info/warn/error 四级

  // V3.2: 类型安全的配置服务
  config: IConfigService; // 读取 manifest.configuration 中声明的设置项

  // V3.2: 声明式贡献点只读视图
  contributions: ContributionAccessor; // list(): 内省插件在 manifest 中声明的贡献点

  // V5.2: 插件 RESTful 路由器（详见 5.15 ctx.http）
  http: IPluginHttpRouter; // get/post/put/patch/delete/stream 端点声明

  // V2.5: 主应用共享模块引用（白名单控制）
  require(moduleName: string): any;
}
```

⚠️ **隔离模式限制**：以上接口为 Inline 模式的完整 API。在 Worker Thread / 子进程（`process`）隔离模式下，`ctx.config`、`ctx.provide()` **均不可用**，`ctx.db.migrate()` 回调仅暴露 `prepare()`，`eventBus` 使用 `subscribe()/unsubscribe()` 而非 `on()/off()`。第三方 ZIP 插件会被强制落到 `process` 模式，因此不要在插件里依赖 `ctx.config`。详见 8.1.1 Inline 与隔离模式的 API 差异。

⚠️ `ctx.pluginId` **的语义**：无论 Inline 还是 Worker 模式，`ctx.pluginId` 都是 `plugins` **表的行 UUID**（如 `019fa0d4-2e31-76d9-...`），**不是** `manifest.id`。需要人类可读的稳定标识时读 `ctx.manifest.id`。该差异直接影响 `ctx.db.table()` 的表前缀，详见 §8.1.1 与 §8.6。

2.5 生命周期状态机

```
INSTALLED ──→ ACTIVATING ──→ ACTIVE ──→ DEACTIVATING ──→ INACTIVE
                                  │                          │
                                  └──── ERROR ←──────────────┘
                                                             │
INACTIVE ──→ ACTIVATING（重新激活）                          │
ERROR ──→ ACTIVATING（重试）          UNINSTALLED ←──────────┘
```

- **INSTALLED**：源码已持久化，尚未激活
- **ACTIVATING**：正在执行 `activate()`（瞬态，Inline 模式超时 5 秒 / Worker 模式超时 60 秒）
- **ACTIVE**：正常运行中
- **INACTIVE**：已停用，可通过 toggle 重新激活
- **ERROR**：激活失败，可重试或卸载

2.6 版本兼容性速查表

开发插件前，先确认目标 OpenLearn 版本。以下特性按版本分组，选择适合你的目标版本。

> **版本说明**：自 v0.2.3 起，宿主 API 版本与平台发行版本统一为同一个号（当前 `0.7.4`，运行时读根目录 `package.json`）。下表"最低版本"即为该统一版本号。

API 特性版本要求：

特性	最低版本	说明
`ctx.log`	0.2.x	结构化日志（debug/info/warn/error），自动注入 pluginId 和 timestamp
`ctx.config`	0.2.x	类型安全的配置读取，配合 manifest.configuration 声明
`ctx.provide()`	0.2.x	向 DI 容器注册自定义服务供其他插件消费
`ctx.require()`	0.2.x	引用主应用白名单共享模块（recharts、jspdf 等）
`ctx.invokeCommand()`（前端）	0.2.x	前端直接调用后端 CommandBus
`teacher.panel` 扩展槽位	0.2.x	教师独立全宽管理面板
`student.fullscreen` 扩展槽位	0.2.x	学生全屏视图/考试模式
`global.setting` 扩展槽位	0.2.x	全局设置页扩展
`anchor:*` 锚点扩展槽位	0.2.6	宿主原生按钮/元素前后插入插件按钮
Worker DB `exec()` 转发	0.3.9	Worker 隔离模式下支持通过 `ctx.resolve(IDatabaseToken)` 执行异步 `exec()`
`ctx.http` RESTful API	0.3.11	内置 RESTful 路由器、网关纵深防御与 Worker RPC 隔离（SDK 3.6.0）

扩展槽位版本可用性一览：

槽位	最低版本	适用角色
`teacher.tab`	0.1.x	教师
`teacher.dashboard.widget`	0.1.x	教师
`student.view`	0.1.x	学生
`student.lesson.tool`	0.1.x	学生
`classroom.tool`	0.1.x	课堂教学
`teacher.panel`	0.2.x	教师
`student.fullscreen`	0.2.x	学生
`global.setting`	0.2.x	管理员
`anchor:*`	0.2.6	教师/学生（按锚点所在宿主界面）

`engines.openlearn` **的填法：填你实际用到的最新 API 的最低版本**，不要一律抄 `>=0.2.5`。安装时 `semver.satisfies(OPENLEARN_VERSION, engines.openlearn)` 为假会直接拒绝安装；反过来写低了不会报错，但插件会在旧版本上装着、用到新 API 时才崩。参照上表：只用到 `ctx.log` / `ctx.config` 填 `>=0.2.3`；用到 `ctx.http` 填 `>=0.3.11`；用到 `contributes` 声明式槽位或前端沙箱行为填 `>=0.7.0`。

⚠️ **版本号说明**：OpenLearn 存在独立的版本号维度，请勿混淆：

版本	当前值	说明
平台发行版本	`0.7.4`	主应用平台发布版本（根目录 `package.json`）
宿主 API 版本	`0.7.4`	`engines.openlearn` 检查所用的版本，运行时读同一份 `package.json`
SDK 版本	`3.7.5`	`@openlearn/plugin-sdk` npm 包版本，独立语义化版本管理

三者不同步：宿主版本随平台发版走 `0.x`，SDK 走自己的 `3.x` 线。`engines.openlearn` 填**宿主版本**，不是 SDK 版本。

2.7 导航页面 vs. 白板组件 — 如何区分？

同一个插件可以注册到不同的 slot，**每个 slot 绑定独立的 React 组件**，无需任何 if-else 分支来区分。

核心规则：

Slot	渲染位置	组件收到的 props	适用场景
`teacher.tab`	侧边栏导航 → 全屏独立页面	`{ renderType: 'panel' }`	管理界面（列表、设置、数据看板）
`teacher.dashboard.widget`	白板内的可拖拽卡片	`{ elementId, lessonId }`	课堂交互组件（编辑、答题、展示）
`classroom.tool`	课堂工具架按钮 & 备课画板组件卡片	—	画板工具栏快捷入口与 `LessonPalette` 组件列表入口，注册后教师可在画板
`student.view`	学生端全屏视图	`{ studentId }`	学生操作界面
`anchor:*`	宿主原生按钮/元素前后（`placement: 'before'/'after'`）	—	在指定按钮前后插入插件按钮，锚点 `gin/anchor-slots.md`

`teacher.tab` 和 `teacher.dashboard.widget` 的关键区别：

1. **容器不同**：`teacher.tab` 是全屏页面（`flex-1 overflow-auto`），适合列表、设置等管理界面；`teacher.dashboard.widget` 是白板内固定尺寸的卡片，适合课堂即时交互。
2. **props 不同**：白板组件额外接收 `elementId`（当前白板元素 ID）和 `lessonId`（当前课节 ID），可用于数据隔离和持久化。
3. **ID 匹配机制与双轨制沙箱渲染**：`teacher.dashboard.widget` 的 `id` 必须与 `manifest.classroomTools[].payload.data.teacherWidgetId` 一致，`PluginCardRenderer` 通过这个 ID 查找对应组件：

```
// manifest.json — classroomTools 声明
{
  "classroomTools": [
    {
      "id": "scratch-editor-tool",
      "commandType": "whiteboard.draw",
      "payload": {
        "type": "plugin",
        "data": "{ \"teacherWidgetId\": \"my-widget\", \"studentWidgetId\": \"my-widget\", \"width\": 960 }"
      }
    }
  ]
}
```

从平台 **v0.7.7+** 起，白板自定义卡片全面支持**双轨制渲染架构**，第三方插件无需进入白名单即可在白板画布中展示原生级卡片：

- **轨 A：声明式沙箱 HTML（双轨制推荐，无需白名单）**：在 `manifest.json` 的 `contributes` 中直接声明 `view`（映射至插件包内的 `storage/*.html`），平台白板渲染器 `PluginCardRenderer` 会自动将其作为严格沙箱 `<PluginSandboxIframeView variant="panel" />` 挂载，第三方 `process` 隔离模式插件可直接使用 HTML + `/plugin-sandbox/view-bridge.js` 编写原生白板卡片，**无需配置** `OPENLEARN_TRUSTED_INLINE_PLUGINS` **白名单**！

```
// manifest.json — 声明式白板挂件
{
  "contributes": {
    "teacher.dashboard.widget": [{ "id": "my-widget", "label": "教师控制卡", "view": "storage/teacher.html"
}],
    "student.view": [{ "id": "my-widget", "label": "学生互动卡", "view": "storage/student.html" }]
  }
}
```

- **轨 B：React 内联组件（仅限受信任插件）**：在 `src/frontend.tsx` 中编写 React 组件并调用 `ctx.ui.registerExtensionPoint`，只有在平台配置了 `OPENLEARN_TRUSTED_INLINE_PLUGINS` 信任名单（或内置核心插件）时才允许以父页面 Inline 形式同源渲染：

```
// src/frontend.tsx — 仅受信任 inline 插件支持
ctx.ui.registerExtensionPoint('teacher.dashboard.widget', {
  id: 'my-widget', // ← 匹配 payload.data.teacherWidgetId
  component: MyWidget,
});
```

典型模式：导航页 = 管理界面，白板组件 = 交互工具。 例如作业中心应在 `teacher.tab` 注册作业列表管理页，在 `teacher.dashboard.widget` 注册具体某次作业的提交/批改面板。两者共享 `elementId` / `lessonId` 即可通过后端 API 打通数据。

> **控制 Dashboard 可见性**：插件**无需**为此声明任何配置键。插件中心的每张卡片上都有「总览」开关，对所有插件一律可用；用户切换后状态存在浏览器 `localStorage`（`openlearn:dashboardVisibility`），关闭即隐藏该插件 `teacher.dashboard.widget` 注册的小部件。详见 5.11 IConfigService。

2.8 使用 AI Skill 快速开发（推荐）

除了手动参考本指南编写代码，强烈推荐使用官方更新的 **OpenLearn 插件开发 Skill（v2.16.1）** 来辅助开发。Skill 是运行在 Antigravity / Claude Code / Codex 等智能编码代理中的专属工具包，深度对齐 OpenLearn V2（平台 `0.7.x`、SDK `@openlearn/plugin-sdk@3.8.0` 与测试套件 `@openlearn/plugin-test-kit`）的内核架构与安全规范，能自动化插件工程的大部分设计与生成流程。

> 🚩 **给 AI Agent 的提示**：Skill 负责根据需求设计架构并生成**规范的代码工程**。第三方插件分发默认落入 `process` 子进程沙箱，前端运行于无 DOM 权限的隐藏 iframe，装完脚手架后请先对照 2.2 三档执行模式 确认能力边界。

安装与配置（支持 GitHub 主仓 / Gitee 镜像 / 离线目录拷贝三通道）：

```
# ① skills.sh 注册表安装（GitHub 主仓网络通畅时推荐）
npx skills add aymwoo/openlearn-skills/openlearn-next-plugin-dev

# ② Gitee 镜像直连安装（国内网络访问 GitHub 受限时首选）
#    skills CLI 会把完整 Git URL 识别为远程仓库克隆，必须写全 .git 地址，避免使用简写
npx skills add https://gitee.com/aymwoo/openlearn-skills.git

# ③ 离线目录拷贝（完全离线或无网络连接时的通用兜底方案）
#    克隆或解压 openlearn-skills 仓库后，直接将技能子目录复制到本地 Agent 的 skills 路径中：
#    git clone https://gitee.com/aymwoo/openlearn-skills.git
#    cp -r openlearn-skills/openlearn-next-plugin-dev ~/.gemini/antigravity-cli/skills/  # Antigravity
#     (Claude Code 对应 ~/.claude/skills/ ; pi agent 对应 ~/.pi/agent/skills/)
```

> 🔄 **Skill 仓库双平台**：主仓 github.com/aymwoo/openlearn-skills（Issue / PR / Release / CI 主入口） | 镜像 gitee.com/aymwoo/openlearn-skills。 镜像由 GitHub **单向同步**，不跑 CI、不发 Release；提 Issue / PR 请只在一个平台开一份。

安装完成后，在与 AI Agent 的对话中提及 OpenLearn 插件开发需求（例如："*帮我开发一个随堂投票抽奖插件，需要注册课堂工具和命令处理器*"），Skill 将会自动识别意图并激活。

Skill 核心能力（全面适配 OpenLearn V2 最新架构）：

能力维度	最新架构适配说明
📖 **权威文档与 SDK 契约**	实时对齐 `@openlearn/plugin-sdk@3.8.0` API，包含强类型 `Token<T>`、`ctx.provide()` 自定义服务共享以及活动生态 `IActivityRegistryToken` 契约。
💬 **结构化交互设计确认**	自动引导确认插件模式（`server-only` / `full-stack` / `frontend-only`）、Worker Thread / Process 沙箱权限、UI 扩展槽位（`teacherTab`, `classroomTool` 等）及表结构。
🏗️ **标准脚手架与代码生成**	自动生成包含 `package.json`、`tsconfig.json`、`src/index.ts` (后端 Worker 逻辑) 和 `src/frontend.tsx` (沙箱桥接入口，非 React 组件) 的标准项目工程。
🛡️ **安全与规范防错**	自动校验 CQRS 三件套模式（`ActionRegistry` → `CommandBus` → `EventBus`）、CapabilityGuard 权限申报、SQLite 增量迁移脚本与 ESM 沙箱导出规范。
🧪 **测试套件集成**	自动生成基于 `@openlearn/plugin-test-kit@3.3.3` 的 Vitest 单元测试桩（支持 `createMockContext()` 工厂）。
📦 **一键打包与发布**	提供 `npx @openlearn/plugin-sdk build` 或仓库自带 `cli.mjs build` 命令行指导，生成经过 Manifest Schema 验证的插件 `.zip` 分发包。

AI 辅助开发标准化工作流：

```
用户提出需求：「帮我开发一个随堂互动小测验插件」
  → Skill 自动载入 @openlearn/plugin-sdk@3.8.0 规格与核心 Token
  → 交互式确认：用途、沙箱权限 (vfs/lesson/db)、扩展槽位 (teacherTab/classroomTool)
  → 选择插件模板 (full-stack / server-only / frontend-only)
  → 生成项目结构与代码 (Action/Command/Event + 桥接式 frontend.tsx)
  → 自动生成 @openlearn/plugin-test-kit 单元测试
  → 执行 npx @openlearn/plugin-sdk build 编译生成 ZIP 扩展包
  → 引导在 OpenLearn V2 后台管理中上传激活并测试
```

支持的标准插件模板：

模板类型	运行环境	生成的核心文件
`server-only`	Node.js Worker Thread	`package.json`, `tsconfig.json`, `src/index.ts`
`full-stack`	Worker Thread + 隐藏 iframe 前端	`package.json`, `tsconfig.json`, `src/index.ts`, `src/frontend.tsx`
`frontend-only`	隐藏 iframe 前端（无可见 UI）	`package.json`, `tsconfig.json`, `src/index.ts`, `src/frontend.tsx`

官方 Skill 资源与存储库：GitHub github.com/aymwoo/openlearn-skills ｜ Gitee 镜像 gitee.com/aymwoo/openlearn-skills

> 💡 **提示**：在开发复杂插件时，直接使用 AI Skill 生成的基础代码天然遵循平台的 5 阶段启动流水线、DI Token 依赖注入与沙箱隔离规范，可提升 5~10 倍的开发效率。

3. 插件结构详解

3.1 Manifest 完整规范

```typescript
interface Manifest {
  id: string; // 唯一标识，推荐格式 @scope/name
  name: string; // 显示名称
  version: string; // SemVer 版本号（如 "1.0.0"）
  main: string; // 入口文件路径（schema 必填；唯一例外：ZIP 安装路径会在校验前对缺失值注入默认 "index.js"，请始终显式声明）
  description?: string; // 描述
  author?: string; // 作者
  engines?: {
    // 引擎版本约束
    openlearn?: string; // 宿主版本范围，如 ">=0.2.5"
  };
  requires: string[]; // 依赖的服务 Token（格式 @openlearn/core:TokenName@^1.0.0）
  optional?: string[]; // 可选依赖
  capabilitiesProposed: string[]; // 申请的权限（如 "lesson:write", "vfs:read"）
  classroomTools?: ClassroomTool[]; // 前端课堂工具声明
  provides?: string[]; // V3.2: 插件对外提供的自定义服务 Token 名称
  configuration?: {
    // V3.2: 声明式配置 schema
    properties: Record<string, ConfigProperty>;
  };
  updateSource?: {
    // SDK 3.4.3（平台 v0.3.10）新增：远端更新源声明
    type: 'github-release' | 'gitee-release';
    repo: string; // 仓库路径，如 "user/repo-name"
  };
}

interface ConfigProperty {
  type: 'string' | 'number' | 'boolean' | 'integer';
  default?: unknown;
  description?: string;
  enum?: string[];
  minimum?: number;
  maximum?: number;
  scope?: Array<'platform' | 'school' | 'class'>; // 缺省只有 platform
}

interface ClassroomTool {
  id: string; // 工具 ID
  name: string; // 工具名称
  icon: string; // 图标（使用 lucide-react icon name，如 "BarChart3"）
  commandType: string; // 关联的命令类型
  payload?: any; // 默认 payload
}
```

3.1.1 远端更新检测（SDK 3.4.3 / 平台 v0.3.10 新增）

在 `manifest.json` 中声明 `updateSource` 字段后，平台插件中心可自动检测远端仓库（GitHub / Gitee）Release 中的新版本，并支持一键热更新。

```
{
  "updateSource": {
    "type": "github-release",
    "repo": "user/plugin-repo-name"
  }
}
```

检测机制：

1. 服务端优先使用 `git ls-remote --tags` 获取所有 semver tag，若不可用则回退到 GitHub/Gitee Releases HTTP API
2. 优先推荐最新稳定版；若无稳定版更新则提示预发布（pre-release）版本
3. Version tag 需遵循 semver 格式（可带 `v` 前缀），如 `v1.2.0`、`2.0.0-beta.1`

下载与安装：

- 服务端优先从 Release Assets 中拉取 `.zip` 更新包直接安装
- 若服务端下载超时（15s），自动切换至浏览器直传安装
- 用户可在插件卡片上点击「检查更新」手动触发检测

> 💡 **提示**：`updateSource` 为可选字段。不声明此字段的插件在点击「检查更新」时会自动回退扫描本地 `v2_plugins/*/manifest.json` 进行对比更新（v0.3.10+）。

3.1.2 静态 RESTful 路由与安全配置声明（v0.3.11 新增）

若插件对外暴露 HTTP API，建议在 `manifest.json` 中声明 `api.routes`，以便宿主安全网关进行静态合规审计与前置 RBAC 拦截：

```
{
  "api": {
    "routes": [
      { "method": "GET", "path": "/public-stats", "auth": false },
      {
        "method": "POST",
        "path": "/records",
        "auth": true,
        "roles": ["teacher", "administrator"],
        "rateLimit": { "max": 60, "windowMs": 60000 }
      }
    ]
  }
}
```

3.1.1 `classroomTools[].payload` 的占位符机制

工具栏按钮被点击时，宿主把 `payload` **序列化后做字符串替换**，再作为命令 payload 下发（`LiveClassroomView` 的 `handleExecuteTool`）：

占位符	替换为
`$lessonId`	当前正在授课的课节 id（无则空串）
`$classId`	当前选中的班级 id（无则空串）

替换发生在 `JSON.stringify(payload)` 之后、`JSON.parse` 之前，所以**嵌套字段里的占位符同样有效**：

```
{
  "commandType": "poll.create",
  "payload": {
    "lessonId": "$lessonId",
    "scope": { "classId": "$classId" }
  }
}
```

由此得到两条实用规则：

1. **按钮拿不到 lessonId、除非你写占位符。** handler 里 `command.payload.lessonId` 为 `undefined` 是最常见的"点了没反应"原因。
2. **占位符只在这条路径生效。** AI Agent 直接调用同一命令时不会替换 —— handler 必须自己校验 `lessonId` 是否为空并给出明确报错，不能默认它一定有值。

   若 `payload` 需要触发的是**插入白板元素**而非命令，形状为 `{ "type": "plugin", "data": "<JSON 字符串>" }` —— 注意 `data` 是**字符串化的 JSON**，不是对象；宿主会往里补 `segmentId`。这类元素最终由 `PluginCardRenderer` 按 `teacherWidgetId` 找组件渲染，**第三方插件没有组件，因此会显示空白占位**。

3.2 PluginContext — 插件上下文的完整 API

插件通过 `activate(ctx)` 接收上下文对象。`ctx.services` 直接提供 10 个内核核心服务，同时还直接提供了内置的日志、自建表数据库与 HTTP 路由器：

功能 / 服务	访问方式	用途
CommandBus	`ctx.services.commandBus`	注册/执行命令
EventBus	`ctx.services.eventBus`	发布/订阅事件
ActionRegistry	`ctx.services.actionRegistry`	注册 AI 工具
Capability	`ctx.services.capability`	权限管理
Process	`ctx.services.processManager`	后台进程
Storage	`ctx.services.storage`	专属键值存储
AI	`ctx.services.ai`	文本与大模型生成
PointsDimension	`ctx.services.pointsDimension`	积分维度注册；Worker / Process 模式为 `null`
PointsLedger	`ctx.services.pointsLedger`	积分账本读写；Worker / Process 模式为 `null`
Roster	`ctx.services.roster`	班级 / 学生只读（需 `management:read`）；核心表的唯一受支持读取通道
HTTP Router	`ctx.http`	**（v0.3.11 新增）注册 RESTful API（GET/POST/PUT/DELETE 等）**
Database API	`ctx.db`	命名空间自建表操作与版本迁移（`migrate`）
Structured Log	`ctx.log`	自动携带 pluginId 与时间戳的结构化日志

通过 DI 解析更多服务（`IDatabaseToken`、`IPluginHostToken` 等）：

```
import { IDatabaseToken } from '@openlearn/plugin-sdk';
const db = await ctx.resolve(IDatabaseToken);
```

3.3 命令-事件-Action 三件套

这是插件开发的核心模式。每个业务功能需要三样东西：

> **能力串只认一份清单**：`capabilitiesProposed` 与 `capabilityRequired` 的取值必须来自 `packages/core/capability/catalog.ts`（SDK 已导出 `CAP` / `CAPABILITIES`）。构建期可调用 `assertValidManifestCapabilities(manifest.capabilitiesProposed)` —— 未知串直接抛错，不会等到运行时才表现为「点了没反应」。平台在插件激活时也会对未知串打 ERROR 日志。

3.3.1 Action 注册（AI Agent 可调用）

```
await actionRegistry.register({
  id: 'my-plugin-action', // 唯一 ID
  commandType: 'myplugin.action', // 对应的命令类型（宿主会自动补 manifest.id 前缀）
  description: '用中文描述此工具的功能和参数',
  // 所需权限（必须逐字取自真实能力表：capability/catalog.ts 的 CAP，或 SDK 导出的 CAP.*；
  // 人类可读版见 docs/reference/plugin-capability-matrix.md）
  capabilityRequired: 'lesson:write',
  isHighRisk: false, // 是否高危（需教师审批）
  // 是否出现在 AI Agent 的工具箱；默认 true。设为 false 后仍可经 HTTP / commandBus 调用
  exposeToAgent: true,
  inputSchema: {
    // JSON Schema（Google GenAI 格式，type 必须大写）
    type: 'OBJECT',
    properties: {
      param1: { type: 'STRING', description: '参数说明' },
      param2: { type: 'NUMBER', description: '参数说明' },
    },
    required: ['param1'],
  },
});
```

3.3.2 Command Handler（业务逻辑）

```
await commandBus.registerHandler('myplugin.action', {
  async execute(command) {
    const payload = command.payload as any;
    const { param1, param2 } = payload;

    // 业务逻辑...
    const result = await doSomething(param1, param2);

    // 发布事件通知其他模块
    await eventBus.publish({
      id: crypto.randomUUID(),
      type: 'myplugin.action_done', // 过去式命名
      source: 'plugin.myplugin', // 来源标识
      payload: { param1, result },
      timestamp: Date.now(),
      correlationId: command.id,
    });

    return { success: true, result };
  },
});
```

3.3.3 Event 发布

事件命名规则：**过去式**，点号分隔，如 `lesson.created`、`assignment.graded`。

```
await eventBus.publish({
  id: crypto.randomUUID(),
  type: 'myplugin.action_done',
  source: 'plugin.myplugin',
  payload: {/* 业务数据 */},
  timestamp: Date.now(),
  correlationId: command.id, // 关联原始命令
});
```

3.3.3 课堂工具下发与白板投射闭环（classroomTools 架构深度解析）

很多开发者和 AI Agent 在开发课堂工具时存在误区：以为前端 Toast 提示「✓ 工具下发成功：工具 [xxx] 已成功在当前白板上部署并投射」就意味着白板画布上必定出现了图元。**实际上，该 Toast 仅代表** `/api/commands` **接收端点返回了 HTTP 200**。真正的白板图元生成取决于命令派发链路。

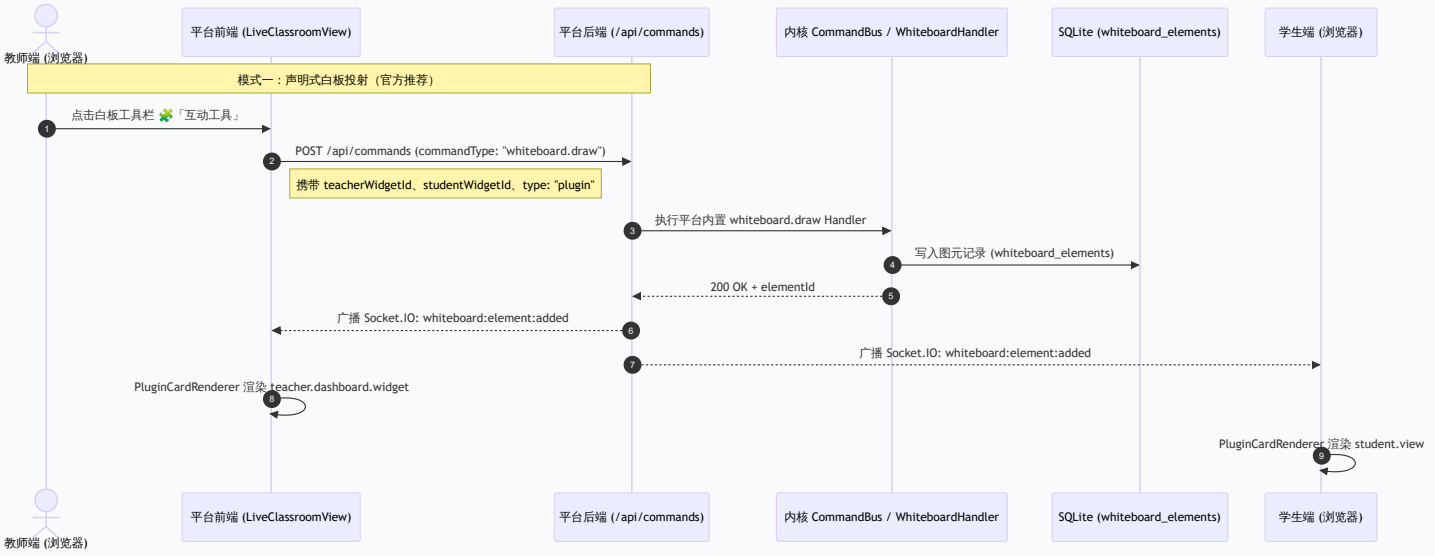
A. 模式一：声明式白板投射（官方推荐标准范式，零后端代码）

如果工具的目标是在白板画布上生成一张卡片挂件（如课堂便签、计时器、随堂测等），**强烈推荐直接使用平台内置命令** `whiteboard.draw`，无需在服务端编写任何 Handler：

```
// manifest.json — 声明式白板卡片下发
{
  "classroomTools": [
    {
      "id": "sticky-note-tool",
      "name": "互动便签",
      "icon": "StickyNote",
      "commandType": "whiteboard.draw",
      "payload": {
        "type": "plugin",
        "data": "{\"teacherWidgetId\": \"sticky-teacher\", \"studentWidgetId\": \"sticky-student\", \"width\":
400, \"height\": 320}"
      }
    }
  ]
}
```

双端挂件 id 的契约（ `teacherWidgetId` **/** `studentWidgetId` **）**

- 两个 id 分别对应 `teacher.dashboard.widget` / `student.view` slot 里注册的组件 `id`；
- **至少声明一个**。只声明一端时，平台在写入白板前会把同一 id 补齐到另一端并打 `WARN`（适用于双端复用同一挂件）；
- 两端都缺，或 plugin 元素的 `data` 不是合法 JSON 时，`whiteboard.draw` 直接报错、不写入图元 —— 不会出现「教师端正常、学生端永久转圈」；
- 建议同时声明 `pluginId`（插件 manifest id）：widgetId 重名时用于消歧与日志定位，缺失只告警；
- 已写入的历史图元若缺 id，前端会渲染「挂件未正确配置」错误态，而不是无限加载。



B. 模式二：私有业务命令（自定义 Handler）

仅当教师点击工具按钮时需要执行复杂的后端计算、跨系统同步或鉴权逻辑时，才使用私有命令（如 `myplugin.launch`）。此时必须配套在 `activate()` 中注册处理器：

```
await commandBus.registerHandler('myplugin.launch', {
  async execute(cmd, ctx) {
    // 执行私有后端业务
    return { ok: true };
  },
});
```

> [!CAUTION] 🚨 严正警示：严禁通过 SQL 直接写 `whiteboard_elements`！子进程沙箱插件（`process` / `worker` 模式）如果试图在自定义 Handler 中通过 `rawDb.prepare("INSERT INTO whiteboard_elements ...")` 手动向白板落库，会被 `sql-guard` **零信任表黑名单强行拦截报错**！inline 插件默认**不走**这道守卫（存量插件兼容），本地自查请设 `OPENLEARN_SQL_GUARD_ALL_MODES=1` —— 打开后 `ctx.resolve(IDatabaseToken)` 的 `prepare` / `exec` / `pragma` 与 worker 同一口径，错误码 `PLUGIN_SQL_REJECTED`。现象为：前端虽弹窗显示「下发成功」（HTTP 200 接收成功），但控制台抛出 SQL-Guard 异常，白板画布空无一物。**正确白板投射方式**：在 `manifest.json` 申请 `whiteboard:write` 权限，在 Handler 中调用 `ctx.services.commandBus.execute({ type: 'whiteboard.draw', payload: { ... } })` 由内核安全代发！

3.3.4 命令必须有 ActionDescriptor（内核 default-deny）

内核的 `CommandBus` 拦截器采用 **default-deny**：只有先通过 `ctx.services.actionRegistry.register()` 登记过 `ActionDescriptor`（含 `capabilityRequired` + `inputSchema`）的命令才允许派发。 **只 `registerHandler()` 而漏了描述符的命令，一律被拒绝**，安全策略不会被绕过。

- **注册顺序**：先 `actionRegistry.register()`，再 `commandBus.registerHandler()`。 顺序反了不会报错，但插件**激活成功后宿主会打印一条 WARN**：

  ```
  [PluginHost] 插件 "<id>" 有 N 个命令 handler 没有 ActionDescriptor：<type-a>, <type-b>。
  这些命令会被 default-deny 以 403 COMMAND_NOT_REGISTERED 拒绝，请为每个 registerHandler 同时注册
  actionRegistry.register(...)。
  ```

  这条日志是激活后的**运行时差集**（激活前后比对宿主 CommandBus 的命令类型），所以 `for` 循环 / 对象表 / `regHandler()` 包装这类静态扫描抓不到的动态注册也能覆盖。 限制：只适用于 inline 插件 —— worker / process 的 handler 在隔离进程内，宿主看不到。

- **自检接口**：`actionRegistry.listMissingDescriptors(commandTypes)` 返回其中没有描述符的命令，`listRegisteredCommandTypes()` 返回已登记的 commandType。插件自带 smoke test 可以直接断言：

  ```
  expect(ctx.services.actionRegistry.listMissingDescriptors(['poll.create', 'poll.vote'])).toEqual([]);
  ```

- **HTTP 表现**：经 `/api/commands` 派发被拒时返回 **HTTP 403**，响应体为 `{ "success": false, "code": "COMMAND_NOT_REGISTERED", "error": "…" }`。 前端应按 `code` 分支处理，不要把它当作 500 服务端崩溃上报。 能力不足被拒时 `code` 为 `COMMAND_CAPABILITY_DENIED`。
- **旧插件迁移**：历史插件若只注册了 handler，升级后其命令会被拒绝。请为每个 命令补上 `ActionDescriptor`（`commandType` 用**你写在源码里的原始命令名**，例如 `poll.create`，不要手写命名空间前缀 —— 宿主会自动加前缀）。 可用 `node scripts/plugin-command-audit.mjs` 列出仓库内所有「有 handler 无 action」的命令。

3.3.5 同一个 commandType，四种位置三种前缀语义

这是继 default-deny 之后第二高频的 `COMMAND_NOT_REGISTERED` 来源：**写短名还是全限定名，取决于写在哪里**。写错不会报错，只在运行期 403。

写在哪里	写什么	宿主是否加 `manifest.id` 前缀
`commandBus.registerHandler(type, …)`、`actionRegistry.register({ commandType })`（及 manifest 里声明的那一份）	短名，如 `poll.create`	✅ 自动加
`manifest.classroomTools[].commandType` / `manifest.contributes.*`	短名，如 `poll.create`	✅ 自动加
运行期 `ctx.ui.registerExtensionPoint(slot, { slotProps: { commandType } })`	**必须全限定** `{{manifest.id}}.poll.create`	❌ 原样透传（`ui.register` 不做前缀化
`commandBus.execute()`	同上一条的对应形态	inline 自动加；worker / process 需全限

判断方法：**短名只在你交给宿主存储/索引的位置生效**（manifest、`registerHandler`）；一旦你自己把命令名写进 `slotProps` 或 `execute()`，就没人替你补前缀了。不确定时一律写全限定名：它在四处都能解析。

---

4. 手把手实例项目

4.1 项目：随堂投票插件

我们将创建一个完整的"随堂投票"插件，教师可以在白板上创建投票，学生提交选票，实时显示结果。

4.1.1 创建数据库表

```
// poll-plugin.ts
export default {
  manifest: {
    id: 'ext-poll',
    name: '随堂投票插件',
    version: '1.0.0',
    main: 'index.js',
    description: '在课堂上创建实时投票，收集学生回答',
    author: 'Your Name',
    engines: { openlearn: '>=0.3.11' },
    requires: [
      '@openlearn/core:ICommandBusService@^1.0.0',
      '@openlearn/core:IActionRegistryService@^1.0.0',
      '@openlearn/core:IEventBusService@^1.0.0',
      '@openlearn/core:IDatabase@^1.0.0',
      '@openlearn/core:IProcessService@^1.0.0',
    ],
    // 命令的 capabilityRequired 必须落在执行角色已有的能力集合内：
    // - lesson:write  → 教师
    // - student:write → 教师 / 学生
    // - lesson:read   → 教师 / 学生
    // 平台不存在 poll:* 能力；发明新 resource 不会给教师/学生授权，命令会 403。
    capabilitiesProposed: ['lesson:write', 'student:write', 'lesson:read'],
    classroomTools: [
      {
        id: 'poll-tool',
        name: '📊 投票',
        icon: 'BarChart3',
        commandType: 'poll.create',
        // $lessonId / $classId 由宿主在点击瞬间替换成当前课堂上下文，
        // 这是 toolbar 按钮唯一能拿到 lessonId 的途径（详见 §3.1.1）。
        payload: {
          lessonId: '$lessonId',
          title: '随堂投票',
          options: ['同意', '不同意', '弃权'],
        },
      },
    ],
  },

  activate: async (ctx) => {
    const commandBus = ctx.services.commandBus;
    const actionRegistry = ctx.services.actionRegistry;
    const eventBus = ctx.services.eventBus;

    // DI 解析数据库访问
    const { IDatabaseToken } = await import('@openlearn/plugin-sdk');
    const db = await ctx.resolve(IDatabaseToken);

    // ── 1. 创建投票表 ──
    await ctx.db.ensureTable(
      'polls',
      `
      id          TEXT PRIMARY KEY,
      lesson_id   TEXT NOT NULL,
      title       TEXT NOT NULL,
      options     TEXT NOT NULL,   -- JSON: ["选项A", "选项B", ...]
      is_active   INTEGER DEFAULT 1,
      created_at  INTEGER NOT NULL
```

```
    );

    await ctx.db.ensureTable(
      'poll_votes',
      `
      id          TEXT PRIMARY KEY,
      poll_id     TEXT NOT NULL,
      student_id  TEXT NOT NULL,
      choice      TEXT NOT NULL,
      voted_at    INTEGER NOT NULL,
      UNIQUE(poll_id, student_id)
    `,
    );

    const pollsTable = ctx.db.table('polls');
    const votesTable = ctx.db.table('poll_votes');

    // ── 2. Action: 创建投票 ──
    await actionRegistry.register({
      id: 'poll-create',
      commandType: 'poll.create',
      description: '在课程中创建一个随堂投票，教师可选择单选或多选模式',
      capabilityRequired: 'lesson:write',
      inputSchema: {
        type: 'OBJECT',
        properties: {
          lessonId: { type: 'STRING', description: '课程 ID' },
          title: { type: 'STRING', description: '投票标题/问题' },
          options: { type: 'STRING', description: '选项列表 JSON，如 ["同意","不同意","弃权"]' },
          mode: { type: 'STRING', description: '投票模式：single_choice 或 multiple_choice' },
        },
        required: ['lessonId', 'title', 'options'],
      },
    });

    // ── 3. Handler: 创建投票 ──
    await commandBus.registerHandler('poll.create', {
      async execute(command) {
        const payload = command.payload as any;
        const pollId = crypto.randomUUID();
        const options = typeof payload.options === 'string' ? payload.options :
JSON.stringify(payload.options);

        db.prepare(
          `INSERT INTO ${pollsTable} (id, lesson_id, title, options, created_at)
                    VALUES (?, ?, ?, ?, ?)`,
        ).run(pollId, payload.lessonId, payload.title, options, Date.now());

        await eventBus.publish({
          id: crypto.randomUUID(),
          type: 'poll.created',
          source: 'plugin.poll',
          payload: { pollId, lessonId: payload.lessonId, title: payload.title },
          timestamp: Date.now(),
          correlationId: command.id,
        });

        return { pollId, message: `投票「${payload.title}」已创建` };
      },
    });
```

```
// ── 4. Action: 发起投票 ──
await actionRegistry.register({
  id: 'poll-vote',
  commandType: 'poll.vote',
  description: '学生对投票进行选择',
  capabilityRequired: 'student:write',
  inputSchema: {
    type: 'OBJECT',
    properties: {
      pollId: { type: 'STRING', description: '投票 ID' },
      choice: { type: 'STRING', description: '选择的选项文本' },
    },
    required: ['pollId', 'choice'],
  },
});

await commandBus.registerHandler('poll.vote', {
  async execute(command) {
    const payload = command.payload as any;
    const voteId = crypto.randomUUID();

    db.prepare(
      `INSERT OR REPLACE INTO ${votesTable}
                (id, poll_id, student_id, choice, voted_at)
                VALUES (?, ?, ?, ?, ?)`,
    ).run(voteId, payload.pollId, command.actorId, payload.choice, Date.now());

    const stats = db
      .prepare(
        `
      SELECT choice, COUNT(*) as count
      FROM ${votesTable}
      WHERE poll_id = ?
      GROUP BY choice
    `,
      )
      .all(payload.pollId);

    await eventBus.publish({
      id: crypto.randomUUID(),
      type: 'poll.vote_cast',
      source: 'plugin.poll',
      payload: { pollId: payload.pollId, stats },
      timestamp: Date.now(),
      correlationId: command.id,
    });

    return { success: true, stats };
  },
});

// ── 5. Action: 查询投票结果 ──
await actionRegistry.register({
  id: 'poll-results',
  commandType: 'poll.get_results',
  description: '查询指定投票的实时统计结果',
  capabilityRequired: 'lesson:read',
  inputSchema: {
    type: 'OBJECT',
    properties: {
      pollId: { type: 'STRING', description: '投票 ID' },
    },
```

```
    },
  });

  await commandBus.registerHandler('poll.get_results', {
    async execute(command) {
      const payload = command.payload as any;

      const poll = db.prepare(`SELECT * FROM ${pollsTable} WHERE id = ?`).get(payload.pollId) as any;
      if (!poll) throw new Error('投票未找到');

      const stats = db
        .prepare(
          `
        SELECT choice, COUNT(*) as count
        FROM ${votesTable}
        WHERE poll_id = ?
        GROUP BY choice
      `,
        )
        .all(payload.pollId);

      return {
        pollId: poll.id,
        title: poll.title,
        options: JSON.parse(poll.options),
        results: stats,
        total: stats.reduce((sum: number, s: any) => sum + s.count, 0),
      };
    },
  });

  // ── 6. 使用结构化日志 ──
  ctx.log.info('Poll plugin activated successfully', { pollTable: pollsTable });
 },

 deactivate: async () => {
  // ctx.db.dropAllTables() 由 PluginHost 自动调用
  console.log('[Poll Plugin] Deactivated');
 },
};
```

4.1.2 在系统设置中安装

1. 进入「系统设置」→「插件中心」
2. 将上述代码粘贴到代码编辑器
3. 点击「安装插件」
4. 在插件列表中找到 `ext-poll`。第三方插件声明的能力**不会自动生效**：先由管理员在审批页确认 `capabilitiesProposed`（写入 `plugin_capability_grants` 表），再点击激活
5. 若跳过审批直接激活，插件仍能激活，但 `plugin:ext-poll` actor 缺少对应能力；由教师/学生发起的命令不受影响（它们按自身角色鉴权）

4.1.3 使用 AI Agent 调用

安装后，AI Agent 自动获得以下工具（工具名是 `ActionDescriptor.id` 对应的 `commandType`，宿主会补 `ext-poll.` 前缀）：

```
poll.create       — 创建随堂投票（实际命令类型 ext-poll.poll.create）
poll.vote         — 学生投票（ext-poll.poll.vote）
poll.get_results  — 查询结果（ext-poll.poll.get_results）
```

教师可以直接对 AI 说：**"在今天的物理课上创建一个投票，问题是'光速是否为宇宙中最快的速度？'，选项为：是、否、不确定"**

4.1.4 这个示例能看到什么、看不到什么

位置	实际表现
画板工具栏「互动工具」	✅ 出现「📊 投票」按钮（名称/图标由 manifest 渲染，宿主负责）
点击按钮	✅ 下发 `ext-poll.poll.create`，`lessonId` 已由 `$lessonId` 填好，写入成功
投票界面（选项列表、实时柱状图）	❌ **看不到**。第三方插件不能提供 React 组件，也不能渲染可见 UI
AI Agent	✅ 三个 Action 已注册，AI 可直接调用并拿到 JSON 结果

想让**学生**看到投票界面，当前只有两条路：

1. 由 AI Agent 代劳（教师下达指令、AI 读取 `poll.get_results` 并把结果讲出来）；
2. 等平台的声明式 slot DSL 落地（manifest 描述视图、宿主用自身组件渲染，见 frontend-sandbox-capability-boundary §4，状态：**待立项**）。

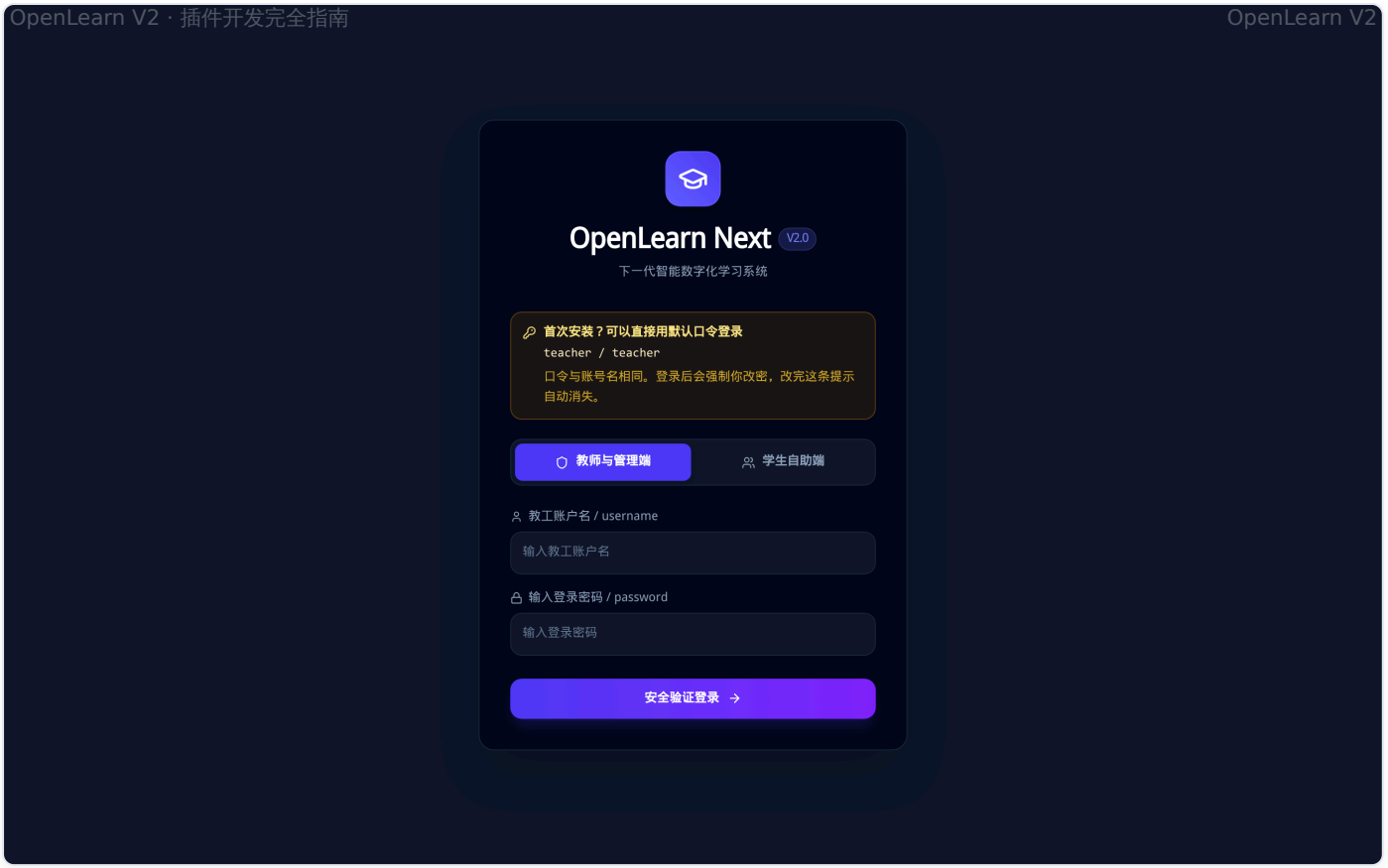
若你的插件必须自带交互界面，那它只能是**内置/受信插件**（放 `packages/plugins` 或进 `OPENLEARN_TRUSTED_INLINE_PLUGINS`），不能走 ZIP 分发。

4.2 第三方插件实装与验收全流程图解（从安装到卸载 9 步法）

为了让开发者能够完整掌握第三方插件的分发、安装、调试与课堂验收全过程，本节基于官方真实实测环境（平台 `openlearn-next v0.7.10+`、`@openlearn/plugin-sdk 3.8.0`、Node 22），提供一套从零到实装验收的**九步图解操作指引**。每一个关键步骤均配有真实的系统操作截图与防错要点。

步骤 1：管理员身份安全登录

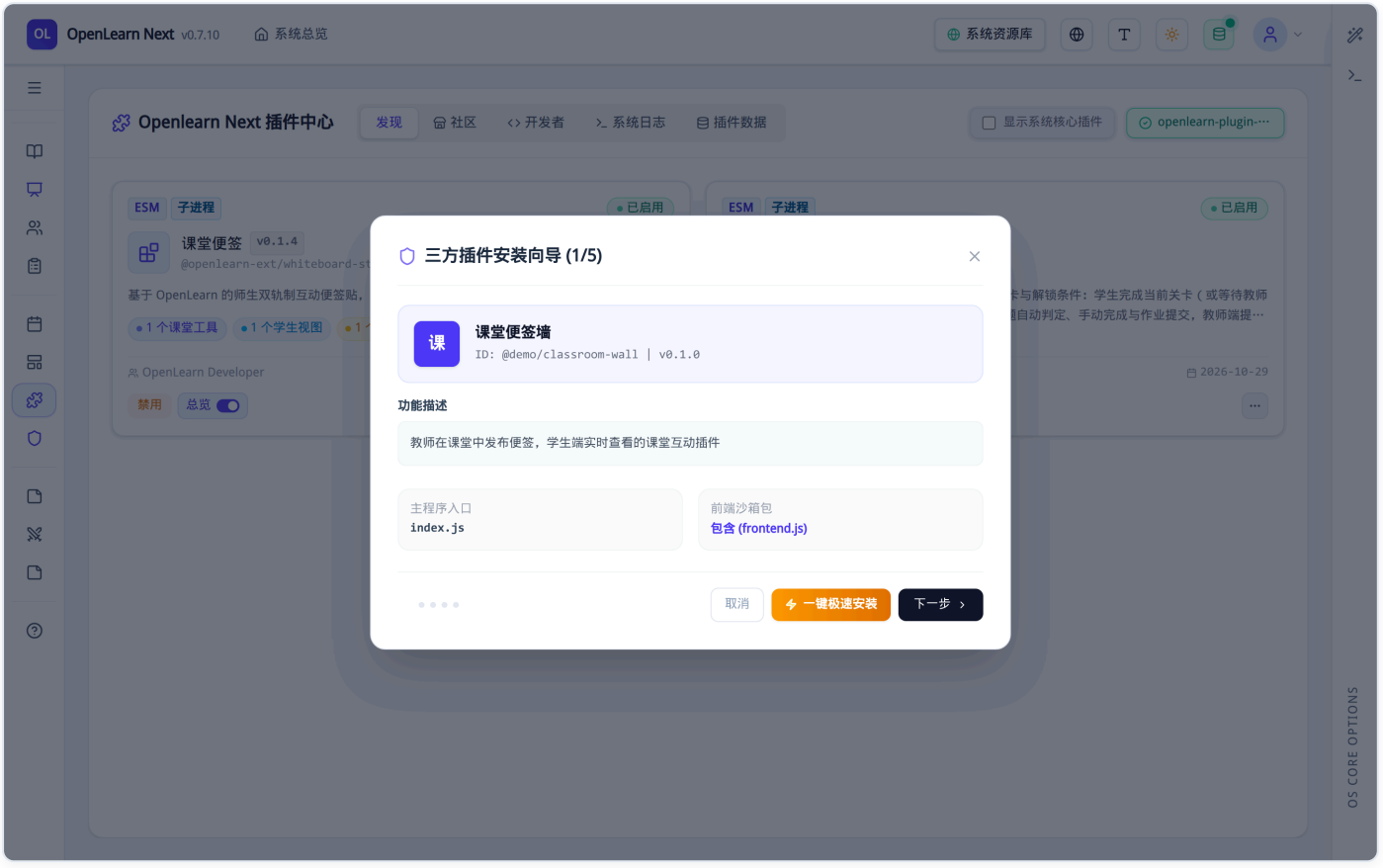
打开平台首页（默认 `http://localhost:9000`），点击「教师与管理端」入口，使用管理员账号登录。



- **安全提醒**：种子管理员账号初始密码等于用户名 `admin`。首次安装系统后登录会触发强制改密（须 ≥8 位且包含字母与数字），改密完成后方可进入系统管理面板。

步骤 2：启动 ZIP 安装向导

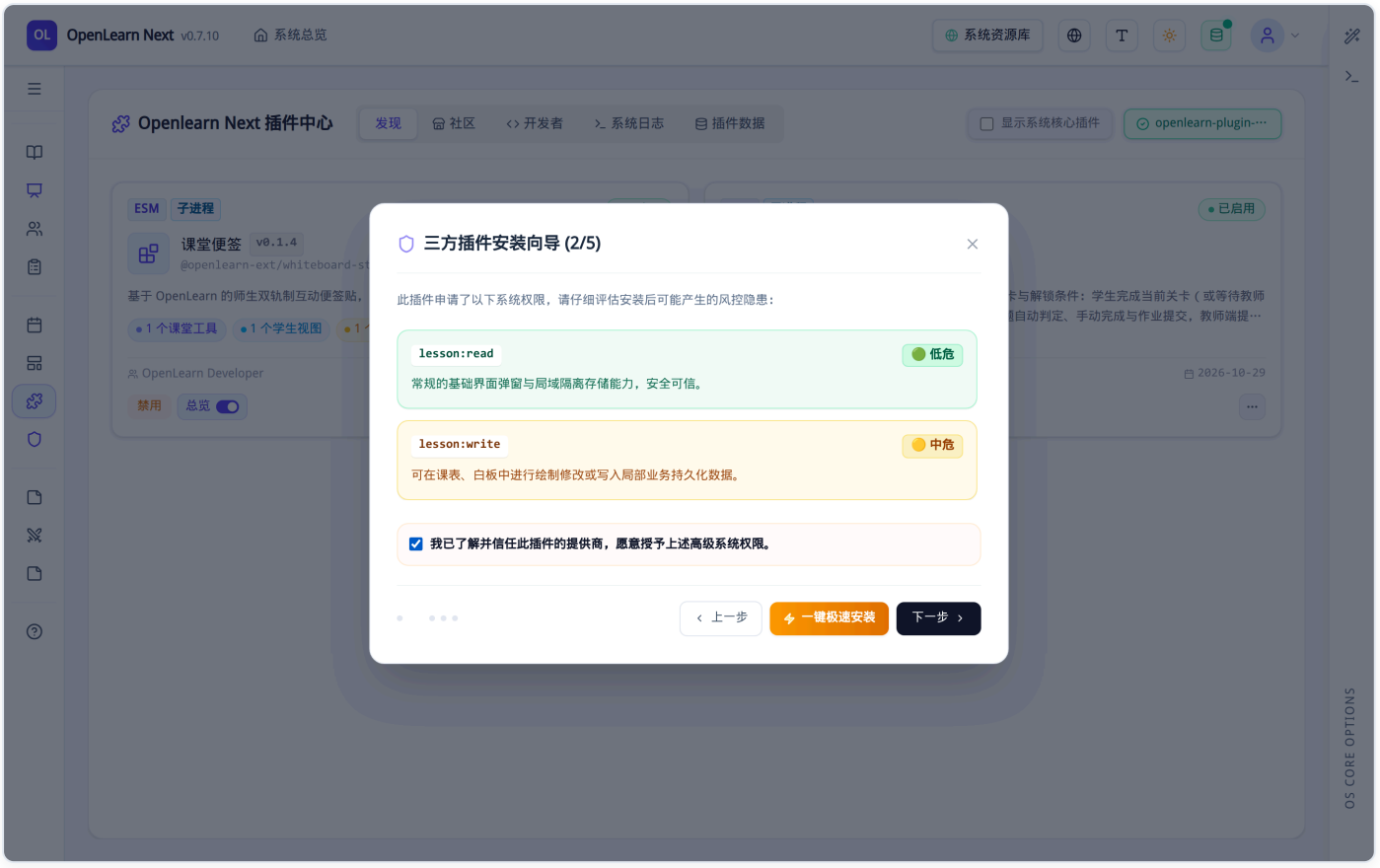
在左侧主导航栏中选择 **系统管理** → **插件中心**，点击右上角的 **从 ZIP 安装** 按钮，选择构建出的 `.zip` 插件包。



- **向导校验**：安装向导包含 5 步流水线，系统会在第 1 步解压 ZIP 并对 `manifest.json` 进行 JSON Schema 严格校验。此时如果 `manifest.id` 或依赖项格式不符合规范，向导会在第 1 步直接标红报错并中断流程。

步骤 3：提供商信任与权限能力审批（关键双重防线）

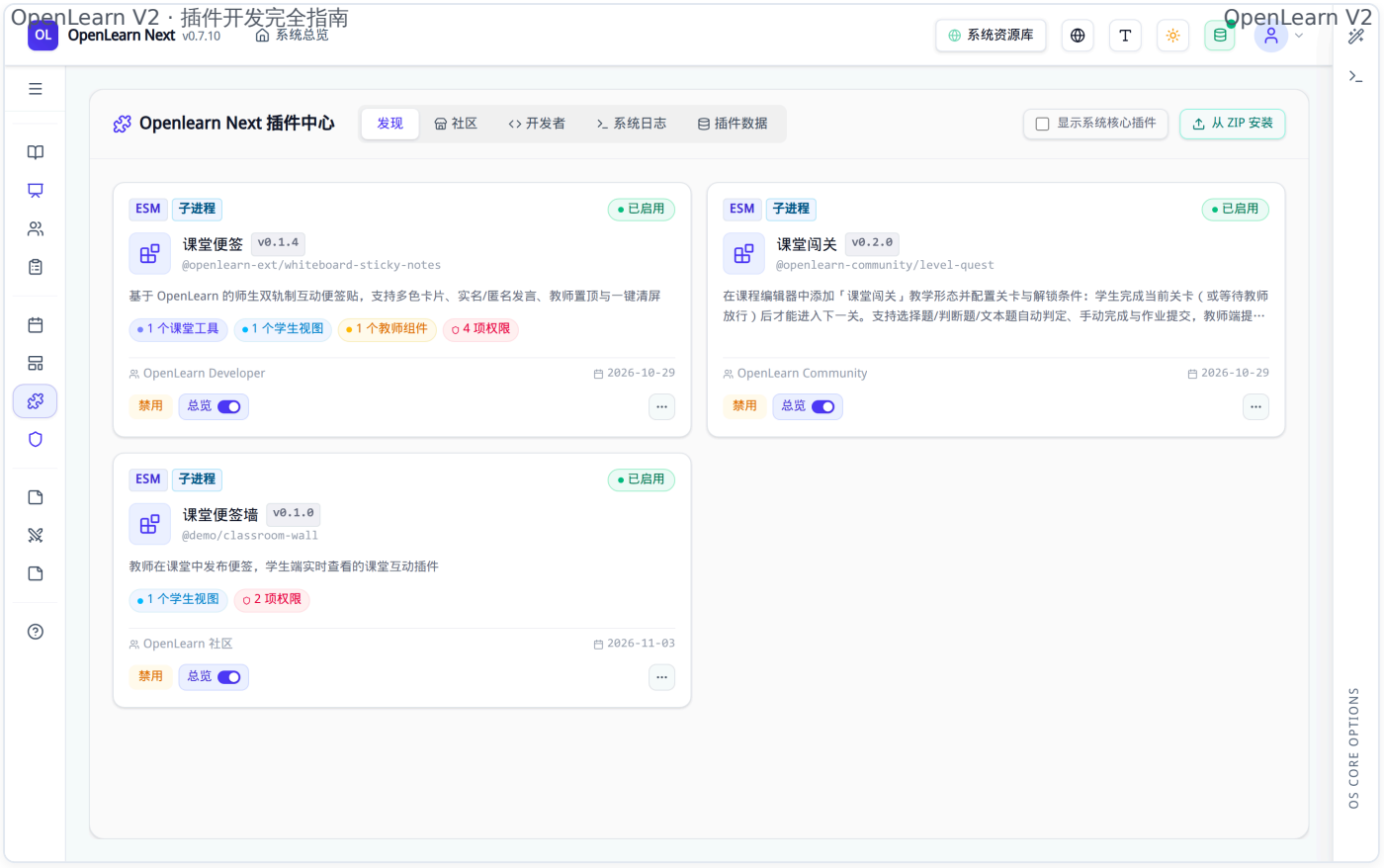
安装向导推进至第 2 步与第 5 步时，管理员必须**显式勾选安全授权确认框**：



- **第 2 步（系统信任确认）**：勾选 `我已了解并信任此插件的提供商，愿意授予上述高级系统权限。`。由于第三方插件强制在隔离子进程（`process` 模式）中运行，该选项确保管理员知悉插件提供商背景；
- **第 5 步（能力授予授权）**：勾选 `确认授予清单申请的能力。不勾选则这些能力不会生效。`。平台采用 Default-Deny 最小权限安全模型，即使 manifest 中声明了 `capabilitiesProposed`，若管理员未在此处勾选，运行时调用对应内核服务依然会被拦截抛出 403。

步骤 4：插件中心状态与隔离模式确认

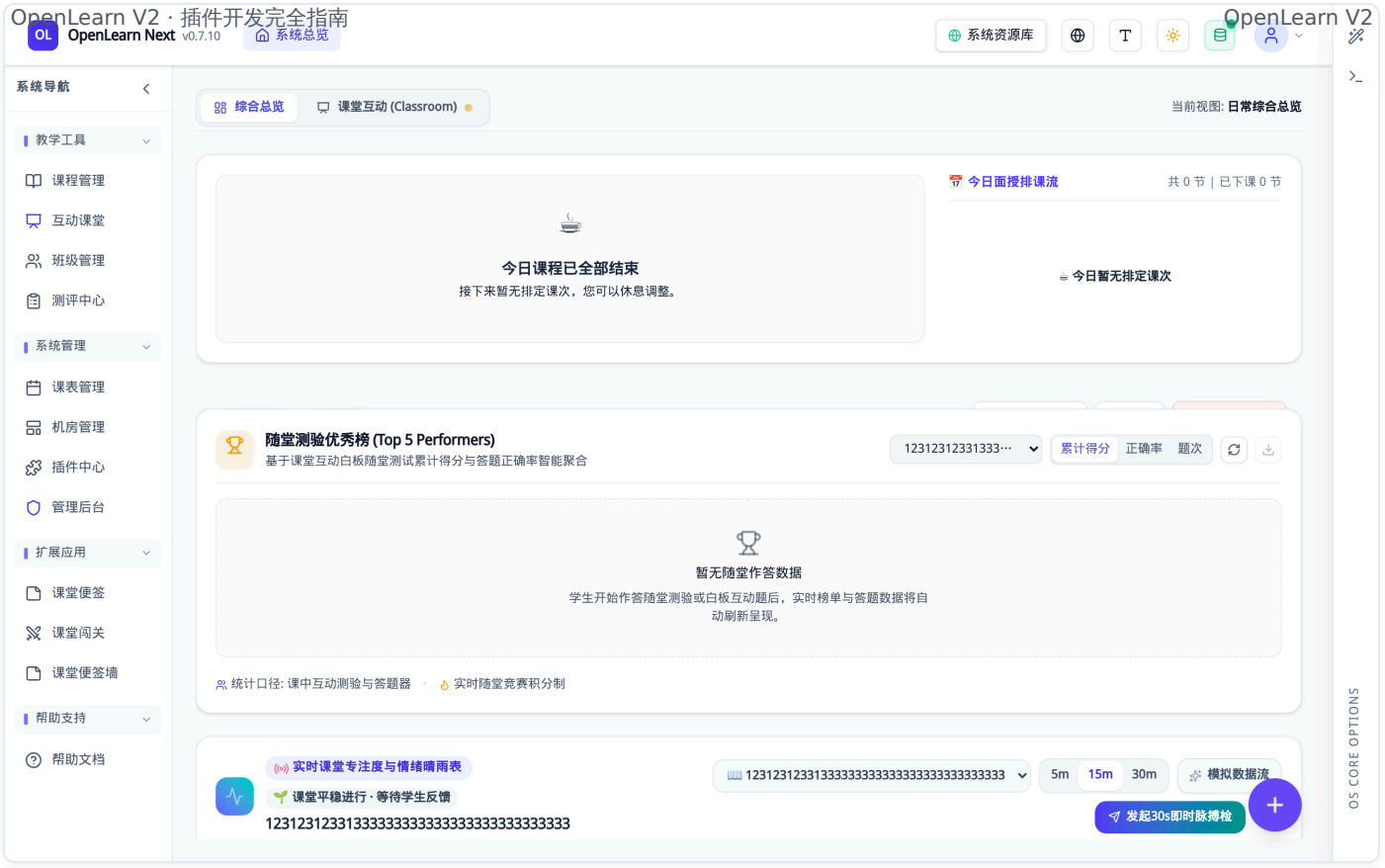
点击 **确认安装并上传** 后，系统完成子进程引导与热加载。返回插件中心列表，对应插件卡片将立即呈现在列表中：



- **卡片要素解读**：
  - **状态徽标**：显示绿色的 `已启用`；
  - **运行模式**：右侧标注 `强制 process`（表明子进程沙箱已就绪，网络出站默认拦截）；
  - **功能注册**：清晰展示注册的视图、扩展槽位与授予的权限数量；
  - **操作按钮**：右下角红色按钮显示 **禁用**，表明当前正处于激活运行中。

步骤 5：教师端扩展导航入口

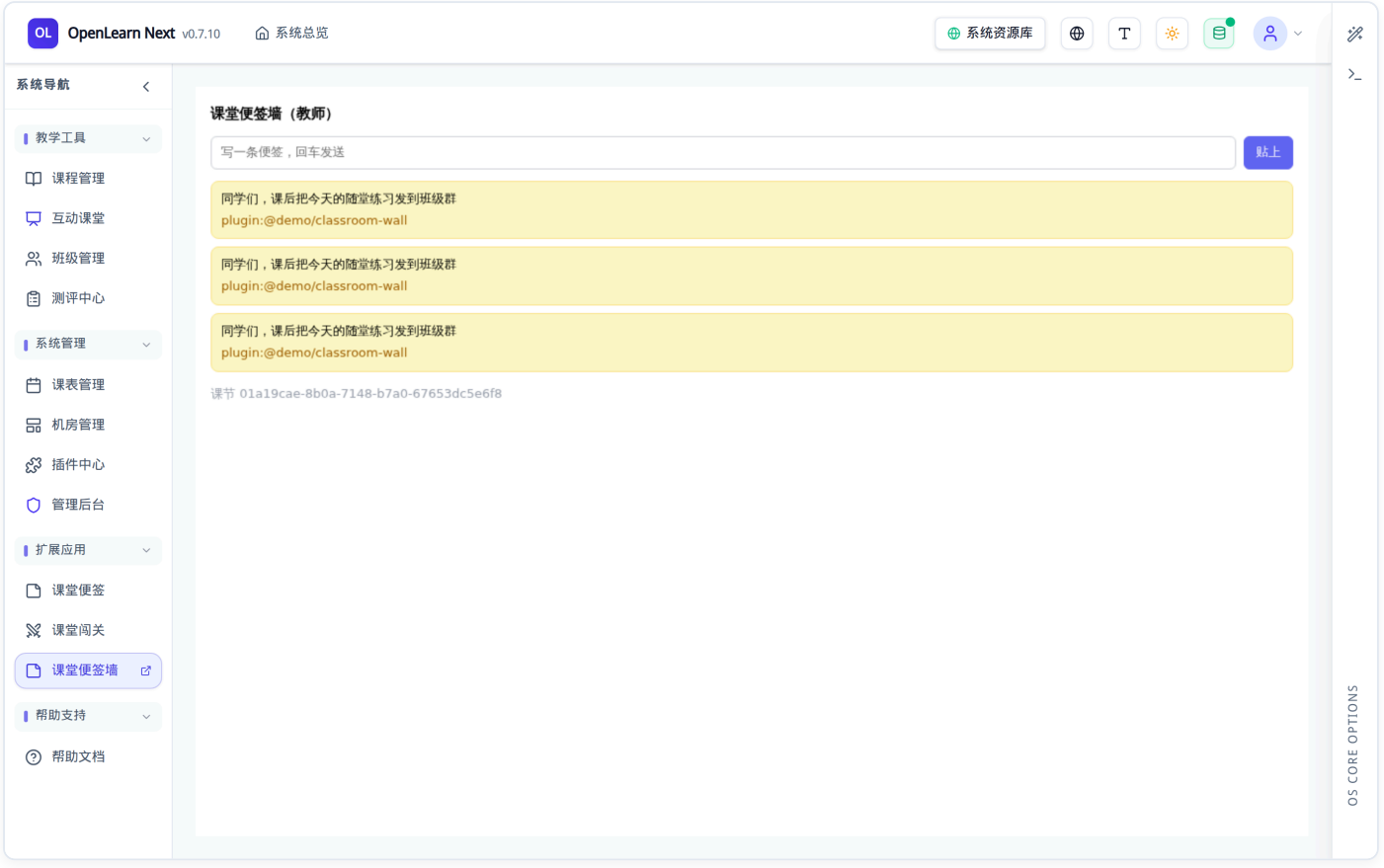
若插件在 manifest 中声明了全屏管理视图（`teacher.tab`），系统左侧导航栏的「扩展应用」分组中会自动挂载该插件的入口：



- **多租户隔离**：导航菜单的显示由教师账号的课堂状态与插件激活状态动态控制，未启用的插件不会产生幽灵菜单项。

步骤 6：沙箱视图运行与安全数据交互

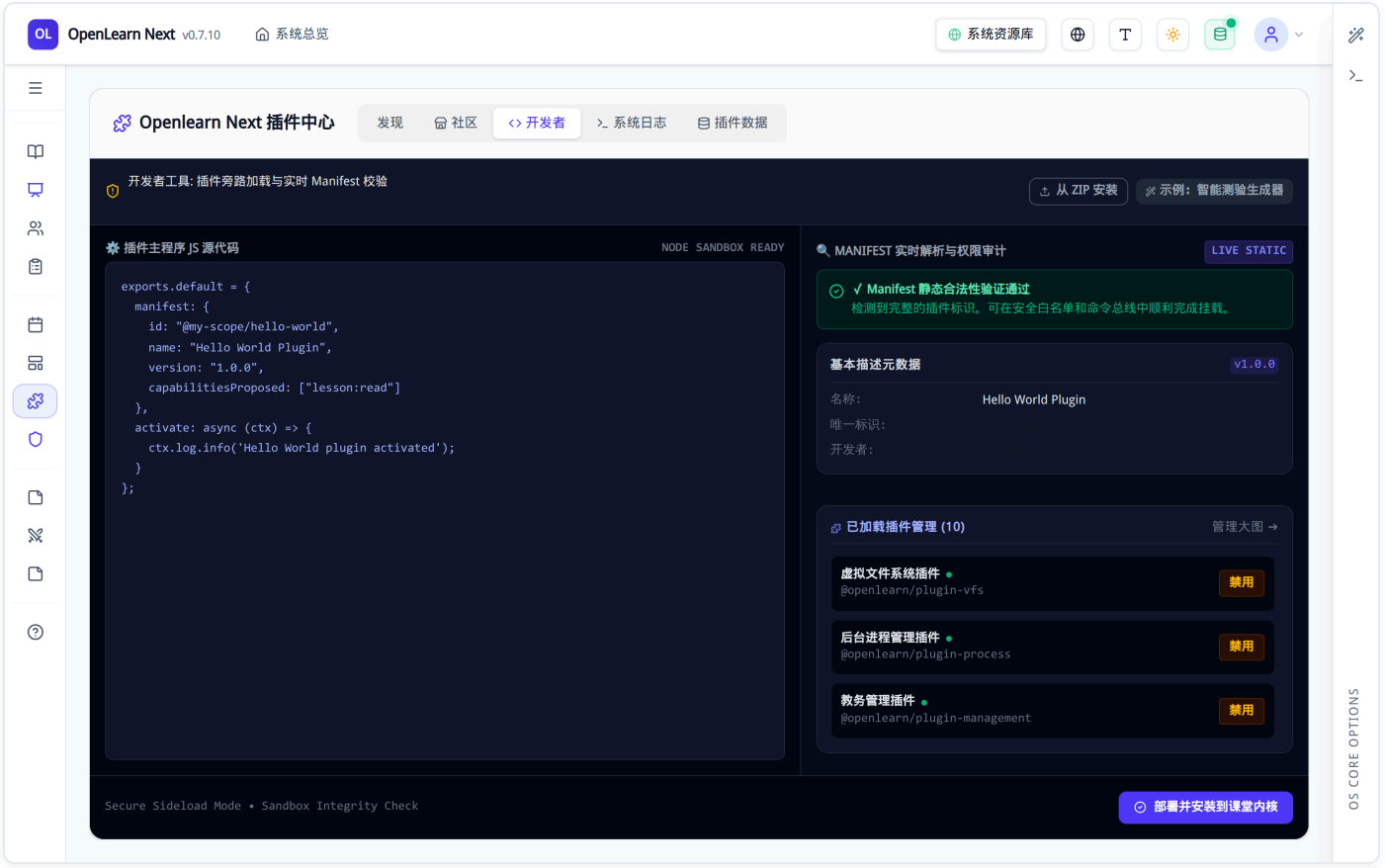
点击导航进入插件页面，第三方插件的视图将在严格隔离的隐藏沙箱 iframe 中渲染：



- **沙箱通信**：第三方插件的前端页面与宿主之间通过 Bridge SDK 建立通信通道，所有对数据库、课节或命令总线的调用均经过平台统一网关与 CapabilityGuard 鉴权。

步骤 7：开发者工具与内核排错观测

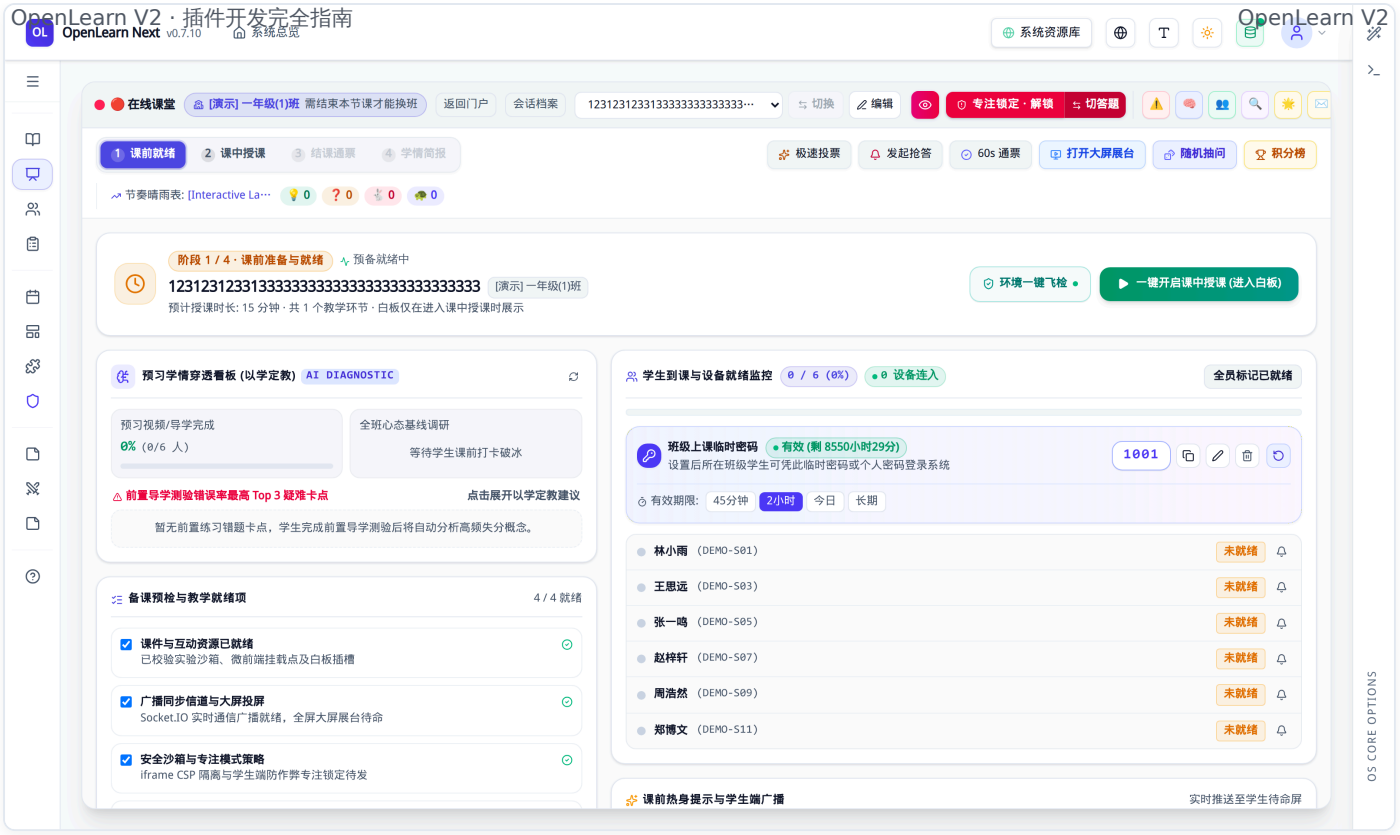
在插件中心点击卡片上的 **开发者** 标签页，可以直观查看插件底层的实时运行指标与日志：



- **排错利器**：在此面板中可实时观测：
  - 插件注册的命令处理器（`registeredHandlers`）；
  - ActionDescriptor 规范与入参 Schema；
  - 事件总线订阅列表与监听状态；
  - 插件子进程输出的实时日志流（包含详细的错误栈），无需登录服务器查看终端日志。

步骤 8：课堂白板现场协同验收

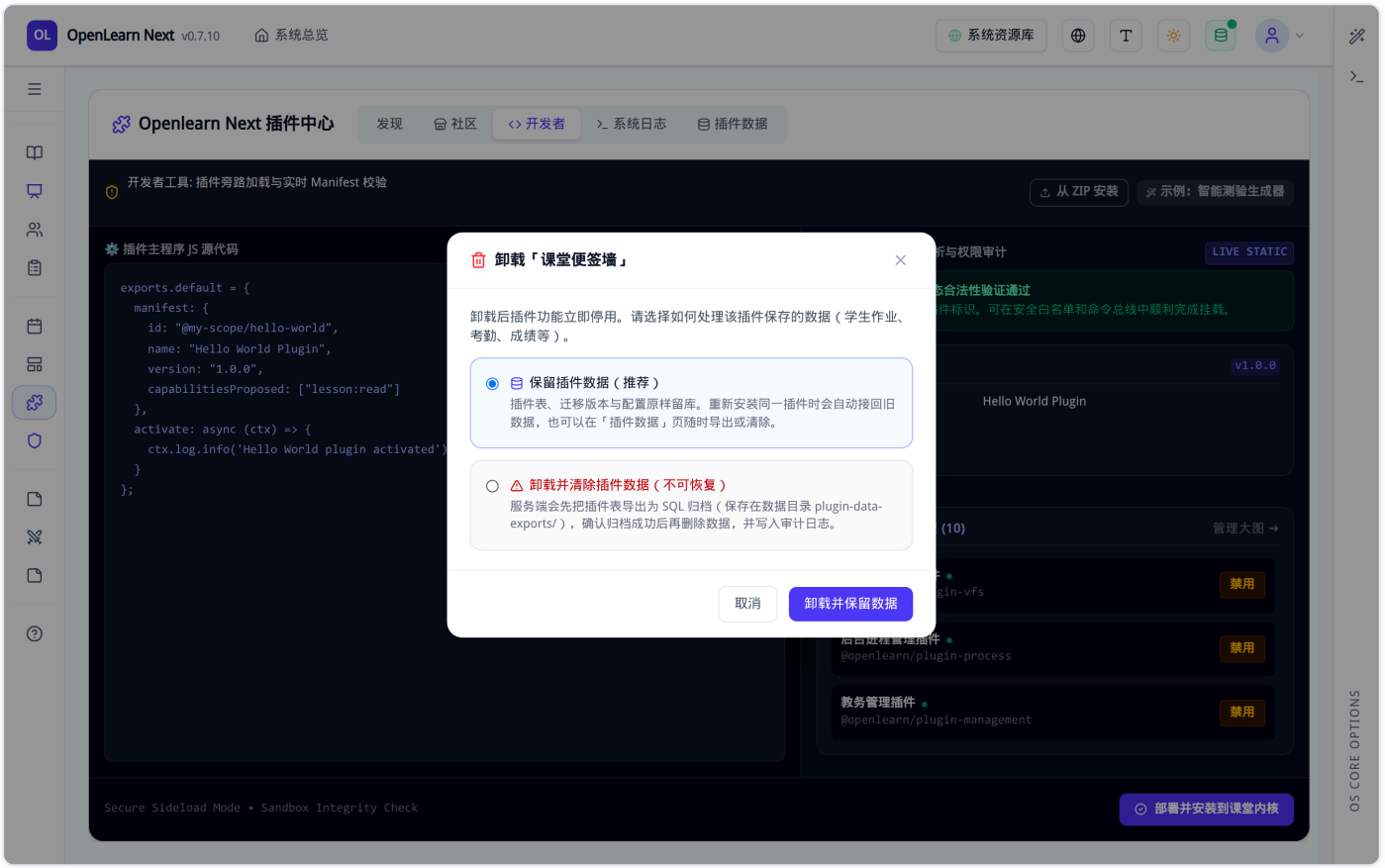
进入 **互动课堂**，选择班级开启上课并点击「一键开启课中授课」，在课中白板工具栏右侧展开 **互动工具**：



- **协同生效**：插件贡献的课堂工具（如投票、抽答、便签墙等）会出现在白板下拉菜单中。点击工具项后，系统自动装配当前课节 ID（`$lessonId`）并向内核命令总线派发指令，右下角弹出成功提示，学生端实时接收同步数据。

步骤 9：插件安全停用与干净卸载

当测试完毕或需要升级维护时，可在插件中心执行停用与安全卸载：



- **资源闭环回收**：

1. 点击卡片右下角的**禁用**，系统触发 `deactivate()` 并向子进程发送终止信号；
2. 点击 **卸载插件** 并二次确认，系统将彻底清除注册的命令、注销事件监听、删除解压的静态资源目录并回收数据库隔离空间，保证系统内核零残留。

---

5. API 及接口文档

5.1 命令定义

```ts
interface PlatformCommand<T = unknown> {
  id: string; // UUID v7
  type: string; // 命令类型，点号分隔如 "lesson.create"
  actorId: string; // 操作者 ID
  payload: T; // 命令载荷
  timestamp: number; // Unix 毫秒时间戳
  metadata?: {
    correlationId?: string; // 关联 ID
    agentDelegated?: boolean; // 是否由 AI Agent 代理
    undoable?: boolean; // 是否可撤销
    [key: string]: unknown;
  };
}
```

5.2 服务 Token（依赖注入）

Token 常量	标识符	返回类型	用途
`ICommandBusServiceToken`	`@openlearn/core:ICommandBusService`	`ICommandBusService`	命令执行、注册
`IEventBusServiceToken`	`@openlearn/core:IEventBusService`	`IEventBusService`	事件发布/订阅
`IActionRegistryServiceToken`	`@openlearn/core:IActionRegistryService`	`IActionRegistryService`	AI 工具注册
`ICapabilityServiceToken`	`@openlearn/core:ICapabilityService`	`ICapabilityService`	权限管理
`IProcessServiceToken`	`@openlearn/core:IProcessService`	`IProcessService`	后台进程
`IStorageServiceToken`	`@openlearn/core:IStorageService`	`IStorageService`	K-V 存储
`IAIServiceToken`	`@openlearn/core:IAIService`	`IAIService`	AI 文本生成
`IDatabaseToken`	`@openlearn/core:IDatabase`	`Database` (better-sqlite3)	直接 SQL 访问
`IPluginHostToken`	`@openlearn/core:IPluginHost`	`PluginHost`	插件主机管理
`ISemesterGradeServiceToken`	`@openlearn/core:ISemesterGradeService`	`ISemesterGradeService`	学期成绩管理

⚠️ **better-sqlite3 版本差异**：`ctx.resolve(IDatabaseToken)` 返回宿主进程的 `better-sqlite3` `Database` 实例。可用 API 取决于宿主安装版本，`exec()` 仅 v9.0+ 可用，建议优先使用 `prepare().run()` / `.get()` / `.all()`。

在 `manifest.requires` 中使用格式：`@openlearn/core:TokenName@^1.0.0`

在代码中解析：

```
import { IDatabaseToken } from '@openlearn/plugin-sdk';
const db = await ctx.resolve(IDatabaseToken);
```

P7-A2 统一插件服务（Unified Plugin Services）

P7-A2 将原本分散在插件宿主内部的能力收敛为一组统一的门面（Facade）服务。插件无需感知内核装配细节，直接经 `ctx.resolve(...)` 即可消费。下列 Token 已在 `@openlearn/plugin-sdk` 中导出，返回类型亦同包提供。

Token 常量	标识字符串	返回类型	用途
`IPluginLifecycleManagerToken`	`@openlearn/core:IPluginLifecycleManager`	`PluginLifecycleManager`	插件的安装 / 卸载
`IPluginDistributionManagerToken`	`@openlearn/core:IPluginDistributionManager`	`PluginDistributionManager`	插件分发：ZIP 上
`IPluginRuntimeCompositionToken`	`@openlearn/core:IPluginRuntimeComposition`	`PluginRuntimeComposition`	统一运行时组合（
`IUnifiedExtensionRegistryToken`	`@openlearn/core:IUnifiedExtensionRegistry`	`UnifiedExtensionRegistry`	扩展点统一注册表
`IPluginCapabilityGatewayToken`	`@openlearn/core:IPluginCapabilityGateway`	`PluginCapabilityGateway`	插件能力网关：列
`ICapabilityRegistryToken`	`@openlearn/core:ICapabilityRegistry`	`CapabilityRegistry`	能力注册表：底层

在 `manifest.requires` 中声明依赖，格式为 `@openlearn/core:TokenName@^1.0.0`，例如：

```
{
  "requires": ["@openlearn/core:IPluginLifecycleManager@^1.0.0",
"@openlearn/core:IPluginCapabilityGateway@^1.0.0"]
}
```

在代码中解析：

```
import { IPluginLifecycleManagerToken, IPluginCapabilityGatewayToken } from '@openlearn/plugin-sdk';

const lifecycle = await ctx.resolve(IPluginLifecycleManagerToken); // 类型: PluginLifecycleManager
const gateway = await ctx.resolve(IPluginCapabilityGatewayToken); // 类型: PluginCapabilityGateway

await lifecycle.uninstallPlugin(pluginId);
gateway.listCapabilities().forEach((c) => console.log(c.id));
```

> 注意：这些统一服务由内核在启动期装配完成，插件侧仅通过 Token 消费，避免在插件代码中直接依赖 `@openlearn/core` 的内部模块。

5.3 ICommandBusService

```
interface ICommandBusService {
  execute<T>(command: PlatformCommand<T>): Promise<unknown>;
  registerHandler(commandType: string, handler: { execute(cmd: PlatformCommand): Promise<any> }):
Promise<void>;
  unregisterHandler(commandType: string): Promise<void>;
  createCommand<T>(type: string, payload: T, actorId: string, metadata?: CommandMetadata):
Promise<PlatformCommand<T>>;
  setInterceptor(interceptor: (command: PlatformCommand) => Promise<void>): Promise<void>;
}
```

5.4 IEventBusService

```
interface IEventBusService {
  publish(event: PlatformEvent): Promise<void>;
  subscribe(eventType: string, subscriber: (event: PlatformEvent) => void | Promise<void>): Promise<void>;
  unsubscribe(eventType: string, subscriber: (event: PlatformEvent) => void | Promise<void>): Promise<void>;
}
```

重要: `subscribe('*', handler)` 可订阅所有事件。事件订阅器在插件 deactivate 时由 ResourceTracker 自动取消。

5.5 IActionRegistryService

```
interface ActionDescriptor {
  id: string; // 唯一 ID
  commandType: string; // 对应命令类型
  description: string; // 对 AI Agent 的功能描述（中文）
  inputSchema: any; // JSON Schema（Google GenAI 格式）
  capabilityRequired: string; // 所需权限
  isHighRisk?: boolean; // 高危操作需审批
}

interface IActionRegistryService {
  register(descriptor: ActionDescriptor): Promise<void>;
  unregister(id: string): Promise<void>;
  getAllActions(): Promise<ActionDescriptor[]>;
  getAgentTools(): Promise<unknown[]>;
  getActionByToolName(toolName: string): Promise<ActionDescriptor | undefined>;
  getActionByCommandType(commandType: string): Promise<ActionDescriptor | undefined>;
  /** 已登记描述符覆盖的 commandType */
  listRegisteredCommandTypes(): Promise<string[]>;
  /** 给定命令里没有 ActionDescriptor 的那些（default-deny 会拒绝） */
  listMissingDescriptors(commandTypes: Iterable<string>): Promise<string[]>;
}
```

5.6 inputSchema 格式规范

遵循 Google GenAI `functionDeclarations` 格式：

```
{
  type: 'OBJECT',
  properties: {
    stringParam:  { type: 'STRING',  description: '字符串参数说明' },
    numberParam:  { type: 'NUMBER',  description: '数值参数说明' },
    boolParam:    { type: 'BOOLEAN', description: '布尔参数说明' },
    arrayParam:   { type: 'ARRAY',   description: '数组参数说明',
                    items: { type: 'STRING' } },
  },
  required: ['stringParam'],  // 必填参数
}
```

5.7 IAIService

```
interface IAIService {
  generateText(
    prompt: string,
    options?: {
      systemInstruction?: string; // 系统指令
      temperature?: number; // 温度 (0-1)
    },
  ): Promise<string>;
}
```

使用示例：

```
const summary = await ctx.services.ai.generateText(`请分析以下学生作业并给出评分：\n${homework}`, {
  systemInstruction: '你是一位教学助手，请用中文回复。',
  temperature: 0.3,
});
```

5.8 IStorageService

```
interface IStorageService {
  get(key: string): Promise<unknown>;
  set(key: string, value: unknown): Promise<void>;
  delete(key: string): Promise<void>;
}
```

底层使用 SQLite `plugin_storage` 表，自动按插件 namespace 隔离。

5.9 PluginDatabaseAPI

```typescript
interface PluginDatabaseAPI {
  ensureTable(tableName: string, schema: string): Promise<void>;
  table(tableName: string): string; // 返回完整表名
  dropAllTables(): Promise<void>;
  migrate(targetVersion: number, upgradeFn: (db: any) => Promise<void> | void): Promise<void>;
}
```

新增 `migrate()` 方法：支持声明式数据库版本迁移，参数 `version` 表示目标版本号，若当前版本低于目标版本则执行 `upgradeFn`。

示例：`ctx.db.table('polls')` 返回带前缀的完整表名。平台已统一收敛表前缀为 `plugin_<scopeId>_`：上传与外部插件统一为 `plugins` 表行 UUID（如 `plugin_019fa0d4_2e31_76d9_8322_ca08f60012a8_polls`），内置插件为规范 scopeId。前缀中所有非 `[A-Za-z0-9_]` 字符一律替换为 `_`。完整规则见 §8.1.1「表前缀语义」。

从 DI 获取原始 Database 实例的方法限制：

`ctx.resolve(IDatabaseToken)` 返回宿主编译的 `better-sqlite3` `Database` 实例。由于宿主可能在较老版本的 better-sqlite3 上运行，建议只使用以下兼容方法：

方法	better-sqlite3 版本要求	说明
`prepare().run()`	全版本	执行单条 SQL
`prepare().get()`	全版本	查询单行
`prepare().all()`	全版本	查询多行
`exec()`	>= 9.0.0	批量执行多条 SQL（**Worker 模式不可用**）
`pragma()`	>= 4.0.0	PRAGMA 语句
`transaction()`	全版本	事务包装

最佳实践：

- 使用 `prepare().run()` 逐条执行，代替 `exec()` 批量执行
- 使用 `ctx.db.ensureTable()` + `ctx.db.table()` 管理表名，不要手动拼接
- 数据库操作优先使用 `ctx.db` PluginDatabaseAPI 封装，仅在需要精细控制时直接操作 Database 实例

5.10 IPluginLogger（V2.5 新增）

```
interface IPluginLogger {
  debug(message: string, meta?: Record<string, unknown>): void;
  info(message: string, meta?: Record<string, unknown>): void;
  warn(message: string, meta?: Record<string, unknown>): void;
  error(message: string, meta?: Record<string, unknown>): void;
}
```

自动注入 `pluginId` 和 `timestamp`，替代传统 `console.log`。示例：

```
ctx.log.info('Activation completed', { handlerCount: 5 });
ctx.log.error('DB migration failed', { error: err.message });
```

5.11 IConfigService（V3.2 新增）

```
interface IConfigService {
  get<T = unknown>(key: string): T;
  getAll(): Record<string, unknown>;
  set(key: string, value: unknown): Promise<void>;
  onChange(callback: (key: string, newValue: unknown, oldValue: unknown) => void): () => void;
}
```

配合 `manifest.configuration` 声明使用，自动应用默认值。示例：

```
// manifest 中声明：
// configuration: {
//   properties: {
//     maxOptions: { type: 'number', default: 10, description: '最大选项数' },
//     enableAnonVoting: { type: 'boolean', default: false },
//   }
// }

const maxOptions = ctx.config.get<number>('maxOptions'); // 10
ctx.config.onChange((key, newVal, oldVal) => {
  if (key === 'maxOptions') {
    ctx.log.info('Config changed', { key, oldVal, newVal });
  }
});
// 注意：onChange 只接受一个回调参数，回调签名 (key, newValue, oldValue)。
// 回调对所有已声明配置键都会触发，需在回调内按 key 过滤。
```

> **关于** `showInDashboard`：这个键**不是**框架识别的保留键，也没有任何运行时实现。插件卡的「总览」开关对每个插件都存在，状态保存在浏览器 `localStorage`，与插件配置无关。在 `configuration.properties` 里声明它只会得到一个普通布尔配置项，既不会解锁开关，也不会控制小部件显隐。

5.12 共享模块 require（V2.5）

插件可通过 `ctx.require()` 引用白名单中的 npm 包，无需自行打包：

```
const recharts = ctx.require('recharts');
```

5.13 ResourceService（V2.5 新增）

插件可通过 Command Bus 直接操作系统资源库（`system_resources` 表），无需自行发起 HTTP 请求。

命令列表：

命令	payload	返回值	说明
`resource.list`	`{}`	`{ resources: [...] }`	列出所有资源（id, name, type, created_at）
`resource.get`	`{ id }`	`{ resource: {...} }`	获取单个资源完整内容（含 content 字段）
`resource.create`	`{ name, type, content }`	`{ success, id }`	创建新资源。type 为 `"html"` 或 `"folder"`
`resource.delete`	`{ id }`	`{ success }`	删除指定资源

调用的能力要求： `resource.create` 和 `resource.delete` 需要 `file:write` 权限，插件须在 `capabilitiesProposed` 中声明。

使用示例：

```
const commandBus = ctx.services.commandBus;

// execute() 只接受 PlatformCommand 对象，不接受 (type, payload) 两个参数。
// 用 createCommand() 构造命令；inline 模式同步返回，Worker 模式返回 Promise，统一 await 即可。

// 查询所有资源
const { resources } = await commandBus.execute(await commandBus.createCommand('resource.list', {},
'system'));

// 创建 HTML 课件资源
const { id } = await commandBus.execute(
  await commandBus.createCommand(
    'resource.create',
    { name: 'my-courseware-v1', type: 'html', content: '<html>...</html>' },
    'system',
  ),
);

// 读取资源内容
const { resource } = await commandBus.execute(await commandBus.createCommand('resource.get', { id },
'system'));
console.log(resource.content);
```

注意：当前 ResourceService 未注册为 ActionRegistry 条目，因此不能通过 AI Agent 的 function call 触发，仅供插件代码内直接调用。

共享模块白名单速查：

```
const pdf = ctx.require('jspdf');
const markdown = ctx.require('react-markdown');
const ExcelJS = ctx.require('exceljs'); // 注意：白名单是 exceljs，不含 xlsx
const icons = ctx.require('lucide-react');
const uuid = ctx.require('uuid');
```

5.14 权限字符串规范

```
格式: {resource}:{action}
示例:
lesson:read — 读取课程
lesson:write — 创建/编辑课程
lesson:delete — 删除课程
whiteboard:read — 读取白板
whiteboard:write — 编辑白板
vfs:read — 读取虚拟文件系统
vfs:write — 写入虚拟文件系统
process:write — 创建后台进程
assignment:write — 编辑作业（AI 规划器）
assignment:read — 读取作业列表/详情
assignment:submit — 提交作业（学生）
assignment:review — 互评（学生）
assignment:manage — 建/改作业、分配互评、终评（教师）
management:read — 读取管理数据
management:write — 写入管理数据

通配符: lesson:* 匹配 lesson:read, lesson:write, lesson:delete
```

5.15 插件 RESTful API 开发实战（v0.3.11 新增）

自 OpenLearn V2 `v0.3.11`（SDK `3.6.0`）起，平台正式支持插件对外导出标准 RESTful API，使插件能够与外部系统、前端交互组件或第三方工具直接进行 HTTP 通信。

1. 统一挂载路径规范

所有插件的 HTTP 路由均统一由平台主安全网关挂载至：

```
/api/plugins/:pluginId/*
```

例如插件 ID 为 `ext-homework-hub`，注册了 `/students/:id/summary` 端点，客户端访问的完整路径为：

```
GET /api/plugins/ext-homework-hub/students/stu_1001/summary
```

2. 在 `activate(ctx)` 中注册路由

插件无需引入任何第三方路由库，直接通过 `ctx.http` 注册路由处理函数：

```typescript
import type { PluginContext, PluginApiRequest } from '@openlearn/plugin-sdk';

export default {
  manifest: {
    id: 'ext-course-quiz',
    name: '随堂测验服务',
    version: '1.0.0',
    main: 'index.js',
    api: {
      routes: [
        // 声明静态安全与 RBAC 规则
        { method: 'GET', path: '/public-info', auth: false },
        { method: 'GET', path: '/questions/:qid', auth: true },
        { method: 'POST', path: '/submit', auth: true, roles: ['student', 'teacher'] },
      ],
    },
  },
  activate: async (ctx: PluginContext) => {
    // 1. 申请公开的端点（auth: false 不会自动放行，需管理员写入 plugin_public_routes）
    ctx.http.get('/public-info', async () => {
      return { status: 'online', plugin: ctx.pluginId, timestamp: Date.now() };
    });

    // 2. 提取动态路径参数 (:qid) 与查询参数
    ctx.http.get('/questions/:qid', async (req: PluginApiRequest) => {
      const questionId = req.params.qid;
      const difficulty = req.query.difficulty || 'normal';

      return {
        id: questionId,
        difficulty,
        title: '计算题：求解未知数 x',
      };
    });

    // 3. POST 请求与身份上下文验证
    ctx.http.post('/submit', async (req: PluginApiRequest) => {
      // 获取经过网关校验的安全身份
      const { actorId, username, role } = req.actor;
      const { answers } = req.body || {};

      if (!answers) {
        return { status: 400, body: { success: false, error: '缺少答题数据' } };
      }

      ctx.log.info('收到答卷提交', { user: username, actorId });

      // 返回标准自定义响应结构
      return {
        status: 201,
        headers: { 'x-quiz-processed': 'true' },
        body: { success: true, submissionId: 'sub_9981', student: username },
      };
    });
  },
};
```

3. 请求与响应 DTO 规范

为保证宿主进程的安全与沙箱隔离，插件绝不直接接触 Node.js 原生 Express `req` / `res` 对象，全部通过类型安全的只读 DTO 进行通信：

- `PluginApiRequest` **核心字段**：
  - `params`：动态路径参数键值对（如 `:studentId` → `req.params.studentId`）
  - `query`：解析后的 URL Query 对象
  - `headers`：经过安全白名单过滤后的请求头（只读）
  - `body`：请求体 JSON 对象（受网关 1MB 硬限制保护）
  - `ip`：客户端真实 IP 地址
  - `actor`：调用方身份实体（`userId`，`username`，`role`，`permissions`）
- **自动包装响应**：
  - Handler 直接返回普通对象或基本类型时，路由器自动将其作为 200 OK 封装为 `{ status: 200, body: 返回值 }`。
  - 需要自定义状态码或响应头时，显式返回 `{ status: 400, headers: { ... }, body: { ... } }`。

4. 平台统一安全网关（`server/routes/plugin-api-gateway.ts`）的六道纵深防御

外部发往插件的每一笔 HTTP 请求，均在主线程网关前置执行强力安全过滤：

1. **系统保留端点避让**：核心管理动作（如 `/config`，`/toggle`，`/contributions`）自动避让至系统路由，防止插件拦截管理能力；
2. **路径遍历防护 (Path Traversal)**：对原始子路径与规范化路径双重检测 `..`，命中直接阻断并返回 `400 Bad Request`；
3. **Payload 体积硬限制**：请求体限制为 **1MB**（超限直接拒绝并返回 `413 Payload Too Large`），有效防范恶意刷大报文造成主线程或 Worker 线程 OOM；大文件必须通过 `IStorageService` 进行切片或流式传输；
4. **内存滑动窗口限流器 (Rate Limiting)**：以「**登录身份优先、匿名回落客户端 IP**」+ 插件 ID + 请求方法 + 端点路径为键执行滑动窗口计数，默认单端点 120 req/min，超限返回 `429 Too Many Requests`；可在 `manifest.json` 的 `rateLimit` 中自定义调优。注意：`/api/plugins/*` 的整体流量走的是**按会话分桶**的 `pluginApiLimiter`（600 req/min/会话，未登录回落 IP），不再占用全站 `writeLimiter` 的额度；
5. **前置认证与细粒度 RBAC 拦截**：支持 Cookie 会话（`edu_os_token`）与 Bearer Token 鉴权。未登录返回 `401 Unauthorized`，角色不符返回 `403 Forbidden`。清单中的 `auth: false` 只是申请公开，管理员还须通过安装参数 `approvePublicRoutes: true`、请求头 `x-approve-public-routes: true` 或 `POST /api/plugins/:id/public-routes` 把该方法与路径写入 `plugin_public_routes` 后，匿名访问才会放行；
6. **响应脱敏清洗**：插件出站响应会被强制清洗，剥离 `Set-Cookie`、`Content-Security-Policy`、`Access-Control-Allow-Origin` 等高危标头，杜绝插件尝试劫持主域 Cookie 或篡改全站安全策略。

5. Worker 隔离模式下的透明 RPC 与看门狗熔断

在 Worker 线程隔离模式下，插件代码运行在独立的子线程中。当外部请求到达时：

- 主线程将 `PluginApiRequest` DTO 序列化后通过 IPC 发送至 Worker 线程；
- Worker 线程在内部的 `PluginHttpRouter` 执行插件 Handler，并将响应结果回传；
- **看门狗 5000ms 超时熔断**：若 Worker 插件陷入死循环或计算阻塞超过 5 秒，网关自动触发熔断机制，直接向客户端返回 `504 Gateway Timeout`，彻底消除由于第三方插件假死拖垮整个 Node.js 宿主连接池的隐患。

6. 前端与课件扩展（分两篇）

前端与课件扩展的细节量已超过「教程」该承担的篇幅，且读者前提不同：一篇讲**代码怎么跑起来**，一篇讲**内容怎么进课堂**。因此本章只留导航：

子指南	讲什么
前端插件指南	`FrontendPluginHost` 三档加载、`FrontendPluginContext`、前端服务接口、UI 扩展槽位、图标与导航栏、`invokeCommand`、`HostS` 与构建排查
课件与教学环节扩展	备课画板图元（Palette Item）、备课环节类型（Segment Type）、教学环节进入条件（Gate）

> 🚩 **读之前先确认档位**：第三方 ZIP 插件的前端跑在**隐藏**的不透明源 iframe 里，**不能提供任何可见 UI**（`component` 是函数，`postMessage` 无法结构化克隆）。 可见交互只能来自 manifest 的声明式贡献，由宿主渲染。 边界见 前端沙箱能力边界，档位判定见 2.2。 两篇里凡标注「仅 inline / worker」的 API，第三方插件都用不了。

7. 安全与权限

7.1 高危操作审批

设置 `isHighRisk: true` 的 Action，AI Agent 执行时会进入审批流程：

```
await actionRegistry.register({
  id: 'dangerous-op',
  commandType: 'lesson.delete',
  description: '删除课程。高风险操作。',
  capabilityRequired: 'lesson:delete',
  isHighRisk: true,  // ← 需要教师审批
  inputSchema: { ... },
});
```

执行流程：

1. AI Agent 调用此工具
2. 命令被写入 `pending_commands` 审批表
3. 教师收到审批通知
4. 教师可选择批准、拒绝或修改参数
5. 批准后才实际执行

注意：`administrator` 角色执行时自动绕过高危审批。

7.2 权限模型

- 插件通过 `capabilitiesProposed` 声明所需权限

- 教师/管理员在安装时可审查权限
- 运行时通过 CapabilityGuard 拦截检查
- 支持通配符匹配（如 `lesson:*` 匹配所有课程操作）

7.3 指令隔离与命名空间保护

为防止第三方插件恶意冒充、拦截或篡改内核及其他插件的敏感指令，OpenLearnV2 实施了 **命名空间防欺骗保护**：

命令解析规则：

1. **系统和内核插件**（`@openlearn/` 前缀）：继承全局命名空间访问权，直接使用全局指令名称
2. **第三方插件指令**：统一自动添加 `{manifest.id}.` 前缀（如 `courseware.query` -> `@courseware-hub/plugin.courseware.query`），防止跨插件指令冲突与越权劫持
3. **前缀显式声明**：若指令名已包含本插件前缀 `{manifest.id}.`，则保持原样，避免重复加前缀

防越权劫持：

- 内核在命令注册阶段自动执行 UUID 强检查
- 第三方插件企图注册以其他非本插件 UUID 格式为前缀的指令时，注册拦截器抛出异常并阻止激活

---

8. 高级特性

8.1 Worker Thread 隔离模式

在生产环境中，插件可在独立 Worker Thread 中运行：

```
// 数据库设置 execution_mode
db.prepare("UPDATE plugins SET execution_mode = 'worker' WHERE id = ?").run(pluginId);
```

Worker 模式的特点：

- 独立线程隔离，崩溃不影响主进程
- 通过 RPC 代理访问内核服务（MethodProxy + EventBusProxy）
- 60 秒激活超时（`ACTIVATE_TIMEOUT_MS` 默认值，可用环境变量 `OPENLEARN_WORKER_ACTIVATE_TIMEOUT_MS` 覆盖，下限 5000ms；Inline 模式为 5 秒 `ACTIVATION_TIMEOUT_MS`）
- 崩溃后自动清理（dispose 强制回收）

8.1.1 Inline 与隔离模式（Worker / 子进程）API 差异 ⚠️

Worker Thread 与子进程（`process`）共用同一套 bootstrap 与 RPC 代理，因此下表的「隔离模式」列对两者都成立。**第三方 ZIP 插件会被强制落到** `process`，开发时不要假设 `ctx.config`、`ctx.provide()` 可用。

API	Inline 模式	隔离模式（Worker / Process）	说明
`ctx.services.commandBus`	完整	`registerHandler` / `execute`	`execute` 调用不自动加命名空间前
`ctx.services.eventBus`	`publish` / `subscribe` / `unsubscribe`	同左（经 `EventBusProxy`）	没有 `on()` / `off()`，SDK 接口只有
`ctx.services.actionRegistry`	完整	`register()`	—
`ctx.http`	完整 `PluginHttpRouter`	完整支持 (GET/POST 等)	**（v0.3.11 新增）** 通过 RPC 跨线程
`ctx.db.migrate(fn)`	`sqliteDb.exec()` 可用	仅 `prepare().run/get/all`	`upgradeFn` 收到受限代理，**无** `exec`
`ctx.db.table()`	返回 `plugin_<scopeId>_` 前缀	同左（前缀与执行模式无关）	两种模式一致，均基于安装主键 sc
`ctx.resolve(IDatabaseToken)`	完整 `better-sqlite3` (同步)	`prepare().run/get/all` + `exec()` (异步)	**（v0.3.9+ 补齐 exec 异步转发）** 无
`ctx.config`	可用	❌ 不可用	需通过 `ctx.manifest.configuration.p`
`ctx.provide()`	可用	❌ 不可用	需 `typeof` 守卫跳过
`ctx.log`	可用	可用	—
`ctx.pluginId`	`plugins` 表行 UUID	`plugins` 表行 UUID	两种模式一致；`ctx.manifest.id` 才
`ctx.manifest`	可用	可用	—

`ctx.db.table()` 表前缀语义（已统一收敛）：

平台已统一收敛表前缀生成规则为 `plugin_<scopeId>_`（统一调用 `pluginTablePrefix(scopeId)`）：

1. 外部上传与社区安装的插件一律使用 `plugins` 表主键 UUID 作为 `scopeId`，在 Inline 与独立进程/Worker 模式下完全一致，保障存储物理隔离与跨模式迁移一致性；
2. 内置核心插件使用规范 scopeId（如 `@openlearn/plugin-vfs` → `plugin__openlearn_plugin_vfs_`）；
3. 前缀中所有非 `[A-Za-z0-9_]` 字符一律替换为 `_`。卸载插件时，宿主依据统一安装主键精准识别并级联清理自建表。

实践建议：不要手工拼接 `plugin_` 前缀表名，一律通过 `ctx.db.table('xxx')` / `ctx.db.ensureTable('xxx', schema)` 获取；同时不要把拼好的表名硬编码进跨模式复用的 SQL 字符串。

关键规则：Worker 中 `commandBus.execute()` **不会**自动给 `type` 加 `manifest.id` 前缀（`registerHandler` 会）。从 Worker 内部调用另一个自己的命令时，必须手动拼接完整 type：

```
// ❌ Worker 中此调用会失败—type 缺少前缀
commandBus.execute({ type: 'myplugin.do_work', payload: {} });

// ✅ 手动加 manifest.id 前缀
commandBus.execute({ type: `${ctx.manifest.id}.myplugin.do_work`, payload: {} });
```

结构化错误类（`packages/core/worker-runtime/errors.ts`）：

- `WorkerActivateError` — 插件在 Worker 内激活失败
- `WorkerTimeoutError` — RPC 调用或激活/停用超时
- `WorkerTransportError` — postMessage 通信层失败
- `WorkerCapabilityError` — 跨边界能力检查拒绝
- `WorkerNotSupportedError` — 运行时不支持的功能

8.2 前端 Worker 模式

前端同样支持 Worker 隔离，通过 `BrowserWorkerManager` 将插件运行在 Web Worker 中，与后端一致的隔离保证。插件需在安装时指定 `executionMode: 'worker'`。

8.3 热重载（开发模式）

在 `NODE_ENV=development` 时，PluginHost 自动启用文件监听：

1. `HotReloadController` 通过 chokidar 监听 `plugins/` 目录
2. 检测文件变更（debounce 300ms）
3. 自动停用旧版本 → 清除中间件 → 激活新版本
4. 无需重启服务器

8.4 生命周期中间件

PluginHost 支持在 6 个生命周期阶段注册中间件（洋葱模型）：

```
pluginHost.registerMiddleware('beforeActivate', async (ctx, next) => {
  console.log(`[Auth] 检查插件 ${ctx.pluginId} 的激活权限`);
  await next(); // 继续执行
});
```

可用阶段：`beforeActivate`、`afterActivate`、`beforeDeactivate`、`afterDeactivate`、`beforeCommand`、`afterCommand`。

8.5 异步后台任务

```
// 注册任务处理器
await processManager.registerHandler('my_task_type', async (processId, payload, state, log, updateState) => {
  log('任务开始...');
  for (let i = 0; i < 10; i++) {
    await new Promise((r) => setTimeout(r, 1000));
    updateState({ progress: i / 10 });
    log(`进度: ${i * 10}%`);
  }
  log('任务完成');
});

// 启动任务
const processId = await processManager.spawn('我的后台任务', 'my_task_type', { input: 'some data' });

// 进程事件
eventBus.subscribe('process.completed', (event) => {
  console.log('任务完成:', event.payload.processId);
});
```

8.6 声明式数据库迁移

使用 `ctx.db.migrate()` 进行插件数据库版本管理：

```
// 首次激活时调用（idempotent）
// ✅ 使用 prepare().run() 同时兼容 Inline 和 Worker 模式
await ctx.db.migrate(1, async (sqliteDb) => {
  sqliteDb
    .prepare(
      `
    CREATE TABLE IF NOT EXISTS my_table (
      id TEXT PRIMARY KEY,
      data TEXT
    )
  `,
    )
    .run();
});

// 后续版本升级
await ctx.db.migrate(2, async (sqliteDb) => {
  sqliteDb.prepare(`ALTER TABLE my_table ADD COLUMN extra TEXT DEFAULT ''`).run();
});
```

⚠️ **Worker 兼容提示**：Worker 模式下 `migrate` 回调接收的是受限数据库代理，仅支持 `prepare().run()` / `prepare().get()` / `prepare().all()`。**不要使用** `sqliteDb.exec()`（仅 Inline 模式可用）。建议始终使用 `prepare().run()` 编写迁移脚本以确保两种模式兼容。

⚠️ **表名前缀不自动添加**：`upgradeFn` 拿到的是**原始 better-sqlite3 实例**（Worker 模式为受限代理），其中手写的表名**不会**被加 `plugin_<id>_` 前缀。上例中的 `my_table` 就是实际表名。若确实需要命名空间隔离，请用 `ctx.db.ensureTable()` / `ctx.db.table()` 建表与拼名。前缀规则见 §8.1.1。

8.7 自定义服务注册（V3.2）

插件可以向 DI 容器注册自定义服务，供其他插件消费：

```
// 插件 A：注册服务
await ctx.provide('@my-scope/IQuestionBank', questionBankService);

// 插件 B：manifest.optional 中声明依赖
// optional: ['@my-scope/question-bank:IQuestionBank@>=1.0.0']

// 插件 B：运行时消费 —— 通过提供方导出的 Token 解析
// ⚠️ Token 是 Token<T> 类实例，ctx.resolve 只接受 Token；
//    形如 ctx.resolve({ name: '@my-scope/IQuestionBank' } as any) 的写法
//    在运行时会失败（Token 名还必须满足 domain:Name 格式，见 8.8）。
const questionBank = await ctx.resolve(questionBankToken);
```

8.8 跨插件服务共享（V3.2）

多个插件可以通过类型安全的 DI Token 互相分享服务。

提供方插件 将接口定义为 Token + Type 对，放在 `src/contracts/` 目录中：

```
// ext-quiz-engine/src/contracts/index.ts
import { Token } from '@openlearn/plugin-sdk';

export interface IQuizEngineService {
  score(answers: Answer[]): number;
  generateQuestion(topic: string): Question;
}

export const QuizEngineToken = new Token<IQuizEngineService>('ext-quiz-engine:IQuizEngineService', '1.0.0');
```

在 `manifest.json` 中声明：

```
{
  "provides": ["ext-quiz-engine:IQuizEngineService"]
}
```

在 `activate` 中提供实例：

```
ctx.provide(QuizEngineToken, new QuizEngine());
```

消费方插件 在编译期导入类型，运行时通过 Token 解析：

```
import type { IQuizEngineService } from 'ext-quiz-engine/contracts';
import { QuizEngineToken } from 'ext-quiz-engine/contracts';

const engine = await ctx.resolve(QuizEngineToken);
// engine 类型为 IQuizEngineService，有完整的 IDE 补全
const score = engine.score(answers);
```

在 `manifest.json` 中声明依赖：

```
{
  "requires": ["@openlearn/core:ICommandBusService@^1.0.0", "ext-quiz-engine:IQuizEngineService"]
}
```

校验机制：

- 安装时：检查提供方 `manifest.provides` 是否声明了 token → warn
- 激活时：检查提供方是否已激活并提供服务 → 阻塞
- 激活顺序：`ext-quiz-engine:IQuizEngineService` 自动推导为对 `ext-quiz-engine` 的依赖，提供方先激活

---

9. 测试与调试

9.1 结构化日志

使用 `ctx.log` 替代 `console.log`，自动注入 `pluginId` 和 `timestamp`：

```
ctx.log.info('Handler registered', { commandType: 'poll.create' });
ctx.log.error('Database connection failed', { error: error.message });
ctx.log.debug('Request processed', { latency: 23, payload: data });
```

9.2 查看进程状态

```
# 查看插件列表
curl http://localhost:9000/api/plugins

# 查看后台进程
# 在应用 UI：系统设置 → 进程管理
```

9.3 事件审计

所有事件自动写入 SQLite `events` 表：

```
SELECT * FROM events WHERE type LIKE 'poll.%' ORDER BY timestamp DESC;
```

9.4 插件测试

使用 `@openlearn/plugin-test-kit` 进行单元测试：

```
npm install --save-dev @openlearn/plugin-test-kit vitest
```

```
// __tests__/index.test.ts
import { describe, it, expect } from 'vitest';
import { createMockContext } from '@openlearn/plugin-test-kit';
import plugin from '../src/index';

describe('my-plugin', () => {
  it('should activate and register handler', async () => {
    const ctx = createMockContext();
    await plugin.activate(ctx);

    // MockCommandBus 公开 handlers Map（key 为命令类型，value 为 CommandHandler）
    expect(ctx.services.commandBus.handlers.has('myplugin.hello')).toBe(true);
  });

  // v0.3.11 新增：测试插件 RESTful API 路由
  it('should respond to GET /api/plugins/my-plugin/status', async () => {
    const ctx = createMockContext({ pluginId: 'my-plugin' });
    await plugin.activate(ctx);

    const res = await ctx.http.handle({
      method: 'GET',
      path: '/status',
      params: {},
      query: {},
      headers: {},
      body: null,
      ip: '127.0.0.1',
      actor: { actorId: 'user:1', role: 'teacher' },
    });

    expect(res.status).toBe(200);
    expect(res.body).toEqual(expect.objectContaining({ online: true }));
  });
});
```

10. 发布前自检清单

在打包插件 ZIP 前，逐项检查以下内容以避免常见问题：

Manifest 检查

- [ ] `id` 用 `ext-name` 或 `@your-scope/name`，全局唯一。**不要用 `@openlearn/` 开头** —— 它既不会让你获得进程内信任（策略层只看白名单与内置目录），又会让运维误以为这是官方包
- [ ] `engines.openlearn` 填**你实际用到的最新 API 的最低版本**（见 2.6），不要一律抄 `">=0.2.5"`：写低了不会报错，但插件会在旧版本上装着、用到新 API 时才崩
- [ ] `requires` 中所有服务 Token 的版本前缀正确
- [ ] `classroomTools` 中每个 `commandType` 都有对应的 `registerHandler`
- [ ] 若导出 HTTP 端点，在 `api.routes` 中声明了端点路径、鉴权要求（`auth`）与角色权限（`roles`）

服务端与构建检查

- [ ] 构建配置中已将 `@openlearn/plugin-sdk` 标记为 `external`，严禁将 SDK 打入插件 ZIP
- [ ] 每个 Action（`actionRegistry.register`）都有对应的 Command Handler（`commandBus.registerHandler`）
- [ ] 每个 Command Handler 都有对应的 Action（`actionRegistry.register`，且先注册描述符再注册 handler）—— 否则内核 default-deny 会以 403 `COMMAND_NOT_REGISTERED` 拒绝该命令（见 §3.3.4）
- [ ] `ctx.db.ensureTable` 创建的每个表都有注释说明用途
- [ ] 跨插件调用（如 VFS）在 `capabilitiesProposed` 中声明了对应权限
- [ ] `activate()` 中所有可能出现异常的操作包裹了 `try/catch`

Worker 兼容性检查 ⚠️（目标运行模式为 Worker 时必查）

- [ ] `ctx.resolve(IDatabaseToken)` 返回的实例在 Worker 中调用 `exec()` 返回 Promise，需 `await`（v0.3.9+）
- [ ] 未在 Worker 中调用同步回调式的 `db.transaction(fn)`
- [ ] 未直接访问 `ctx.config`——改用 `ctx.manifest.configuration.properties` 读取默认值
- [ ] `ctx.provide()` 调用包裹了 `typeof (ctx as any).provide === 'function'` 守卫
- [ ] `eventBus` 使用 `subscribe()/unsubscribe()`，而非 `on()/off()`
- [ ] Worker 内部 `commandBus.execute()` 的 `type` **手动拼接了 `ctx.manifest.id` 前缀**
- [ ] Worker 内 HTTP API 响应体大小受 1MB 硬限制，大文件传输已引导至 `IStorageService`

前端检查

- [ ] `tsconfig.json` 中 `jsx` 为 `"react"`（非 `"react-jsx"`）
- [ ] **内联插件**：`react` / `react-dom` / `recharts` / `lucide-react` 标记为 `external`（由 HostSharedDeps 提供）
- [ ] **第三方 ZIP 插件**：不要为"渲染 UI"而打包 React —— 沙箱 iframe 里 `import React` 求值为 `undefined`，组件也传不进宿主。前端只留 `activate(ctx)` + 数据/命令调用（见 前端插件指南）
- [ ] `frontend.tsx` 的 `default export` 是 `{ activate, deactivate }` 对象，不是组件（沙箱加载器会检查 `typeof plugin.activate === 'function'`）
- [ ] 不直接 import React hooks 之外的 React 子路径（如 `react/jsx-runtime`）
- [ ] 使用的扩展点槽位在目标系统版本中存在（参考 2.6 版本兼容性速查表）
- [ ] 前端调用的命令在服务端有对应的 handler

构建检查

- [ ] `npx @openlearn/plugin-sdk build` 无报错

- [ ] ZIP 产物**根目录**包含 `manifest.json` + `index.js` +（可选）`frontend.js`（注意 `zip -r my-plugin.zip my-plugin/` 会产生嵌套目录导致安装失败，见 11.2）
- [ ] 解压 ZIP 后检查 `index.js` 不包含禁用的裸导入（参考 11.4 常见打包错误与排查）
- [ ] 解压 ZIP 后检查 `frontend.js` 不包含 `import ... from "react/jsx-runtime"`

---

11. 发布与分发

11.1 使用 CLI 脚手架

`@openlearn/plugin-sdk` 自带 CLI（当前 `3.7.5`，已发布到 npm），三个子命令：`init` / `build` / `sign`。

```
# 1. 脚手架（--template 可选：server-only | full-stack | frontend-only，默认 server-only）
npx @openlearn/plugin-sdk init --name demo-vote --template server-only

# 2. 装依赖（CLI 的 build 依赖本地 esbuild / jszip）
cd demo-vote && npm install

# 3. 构建 ZIP
npx openlearn-plugin-sdk build              # 等价于 npx @openlearn/plugin-sdk build
npx openlearn-plugin-sdk build --watch      # 开发期监听

# 4. 可选：Ed25519 签名（社区注册表分发需要）
npx openlearn-plugin-sdk sign <zip> --key ./private.pem --publisher your-name
```

产物（`build` 之后）：

- 目录：`dist/` 下有 `index.js`、`frontend.js`（仅 full-stack/frontend-only）、以及 `<包名>.zip`；
- ZIP 同时在**项目根目录**放一份 `<包名>.zip`；
- 包名取 `package.json` 的 `name`。所以 `--name demo-vote` 得到的是 `openlearn-plugin-demo-vote.zip`，**不是** `demo-vote.zip`。CI 里别硬编码文件名，用 `dist/*.zip` 或读 `package.json`。
- ZIP 内部结构固定为根级 `index.js` + `manifest.json` +（可选）`frontend.js`；`manifest.json` 由 CLI 从打包后的代码里抽取 `export default { manifest }` 生成，**你不需要手写 manifest.json**（手写也支持，CLI 会优先用根目录的 `manifest.json`）。

三种模板的产出：

模板	生成文件	`manifest.id` 生成规则
`server-only`	`package.json`、`tsconfig.json`、`src/index.ts`	`@<author>/<name>`
`full-stack`	上述 + `src/frontend.tsx`、`src/contracts/index.ts`	同上
`frontend-only`	上述前端部分	同上

✅ `frontend.tsx` 的导出契约自 `@openlearn/plugin-sdk@3.7.1` 起已修正（该版本已发 npm）：模板直接生成 `export default { activate, deactivate }` 的**桥接入口**（隐藏 iframe 内的无 UI 执行体），不再生成 React 组件 —— 后者会被沙箱加载器判为 `missing activate function`。 仍用 3.7.0 请手工改这一行，或升级：`npm i -D @openlearn/plugin-sdk@3.8.0`。

⚠️ **不要停在 3.7.1**：它修好了模板，但产物里残留 `node:` 内置模块静态导入，插件打完包运行期就失败， 已在 npm 弃用。模板修复在 3.7.4 中完整保留，直接装 3.7.5。 细节见 Scaffold CLI §3.1。

⚠️ **仍需你自己确认的两处**：

1. `manifest.id` **的 scope**：传了 `--author` 是 `@<author-slug>/<name>`；不传则回落默认作者。 建议改成 `ext-<name>` 或 `@your-scope/<name>` —— 别用 `@openlearn/`，那不会带来任何信任。 `manifest.id` 同时是命令命名空间前缀（`@scope/name.some_command`）。
2. `engines.openlearn` 默认 `>=0.2.5`，按你用到的 API 调高。

11.2 手动打包为 ZIP

```
# 插件目录结构
my-plugin/
  index.js          # 入口 (export default { manifest, activate })
  package.json      # 可选
  README.md         # 文档

# 打包 — 关键：manifest.json 必须位于 ZIP 根目录
# ❌ 错误：以下命令会产生 my-plugin/manifest.json 嵌套结构，
#    安装时报 "ZIP package is missing manifest.json"
# zip -r my-plugin.zip my-plugin/

# ✅ 正确：进入插件目录打包，manifest.json / index.js 位于 ZIP 根
(cd my-plugin && zip -r ../my-plugin.zip .)
```

安装期 ZIP 硬约束（`packages/core/esm-loader/install-utils.ts`）：

- `manifest.json` 必须在 ZIP **根目录**（`zip.file('manifest.json')` 取不到即失败）；
- 全部文件解压后总大小 ≤ **300MB**（ZIP bomb 防护）；
- 禁止路径穿越条目（含 `..` 或以 `/` 开头的路径）；
- 入口文件按 `manifest.main` 读取；若 `main` 以 `dist/` 开头且 ZIP 中不存在，会回退尝试去掉前缀后的路径。

11.3 安装 ZIP 插件

在「系统设置」→「插件中心」上传 ZIP 文件，或使用 **一键极速安装（Express Install ⚡）** 按钮启动向导，系统自动：

1. 解压 ZIP
2. 提取 index.js 作为入口
3. 解析 manifest
4. **使用 esbuild 的** `openlearn-token-enforcer` **插件进行二次扫描**（见 11.4）
5. 验证依赖（SemVer 兼容性检查）

6. 检查权限请求，并通过颜色编码（玫瑰色/高危、琥珀色/中风险、翡翠绿/安全）向用户提示安全风险
7. 存入数据库
8. 可选：立即激活

11.4 常见打包错误与排查

错误： `Import of "<module>" is not allowed`

错误信息示例：

```
Build failed with 1 error:
<stdin>:6:27: ERROR: [plugin: openlearn-token-enforcer]
Import of "crypto" is not allowed.
Plugins may only use relative imports or @openlearn/* Token services.
```

原因： OpenLearn 在接收到上传的 ZIP 后，会使用内置的 esbuild `openlearn-token-enforcer` 插件对入口文件（`index.js`）进行安全扫描。该扫描器**只允许两类导入**：

允许	示例
相对路径导入	`import foo from './utils'`
`@openlearn/*` Token 服务	`import { IDatabaseToken } from '@openlearn/plugin-sdk'`

所有其他**裸 specifier 导入**（包括 Node.js 内置模块）都会被拒绝：

```
// ❌ 禁止 — 会触发 token-enforcer 报错
import { randomUUID } from 'crypto';
import { readFileSync } from 'fs';
import path from 'path';
import _ from 'lodash';
```

解决方案：

① 替换 `crypto.randomUUID()`（最常见）

```
// ❌ 错误写法
import { randomUUID } from 'crypto';
const id = randomUUID();

// ✅ 正确写法 — 使用全局 Web Crypto API，Node.js 20+ 和现代浏览器均可用，无需 import
const id = globalThis.crypto.randomUUID();
// 或更简短（全局 crypto 在 Node.js 20+ 中与 globalThis.crypto 等价）
const id = crypto.randomUUID();
```

② 替换时间戳 ID（更简单）

```
// 对于不需要密码学安全性的 ID，可以用时间戳 + 随机数组合
const id = `${Date.now()}_${Math.random().toString(36).slice(2, 8)}`;
```

③ 替换 `fs`（文件操作）

插件不应直接使用 `fs` 模块，应通过 VFS 服务（需在 manifest 中声明 `vfs:read` / `vfs:write` 权限）：

```
// ❌ 错误写法
import { readFileSync } from 'fs';

// ✅ 正确写法 — 通过 VFS 服务
const { IStorageServiceToken } = await import('@openlearn/plugin-sdk');
const storage = await ctx.resolve(IStorageServiceToken);
await ctx.services.commandBus.execute(
  ctx.services.commandBus.createCommand('vfs.read_file', { path: '/my/file.txt' }, ctx.pluginId),
);
```

④ 替换 `path`（路径操作）

```
// ❌ 错误写法
import path from 'path';

// ✅ 正确写法 — 使用原生字符串操作
const filename = filePath.split('/').pop() ?? '';
const ext = filename.split('.').pop()?.toLowerCase() ?? '';
```

⑤ 替换第三方 npm 包

部分常用包已通过 `ctx.require()` 白名单共享（无需 import）：

包名	使用方式
`uuid`	`const { v4: uuidv4 } = ctx.require('uuid')`
`exceljs`	`const ExcelJS = ctx.require('exceljs')`
`recharts`	`const { LineChart } = ctx.require('recharts')`
`jspdf`	`const { jsPDF } = ctx.require('jspdf')`
`lucide-react`	`const { BookOpen } = ctx.require('lucide-react')`

对于其他第三方包，需在构建阶段将其完整代码内联到 `index.js` 中（esbuild bundle），避免在产物中残留裸 specifier 导入语句。

自检方法（上传前验证）：

```
# 检查产物 index.js 中是否有不在白名单内的裸导入
grep -E '^import .+ from "[^@\./]' dist/index.js
# 有输出 = 有问题；无输出 = 通过
```

11.5 版本兼容性

插件依赖声明支持 SemVer 范围：

- `^1.0.0` — 兼容 1.x.x
- `~1.2.0` — 兼容 1.2.x
- `>=1.0.0 <2.0.0` — 显式范围

安装时 PluginHost 自动检查兼容性，不兼容则拒绝安装。

12. Web 课件与 Bridge SDK 集成

OpenLearn V2 支持嵌入交互式 HTML/Web 课件。为了确保平台安全性，课件运行在高度隔离的 iframe 沙箱中（设置了 `sandbox="allow-scripts allow-forms allow-downloads"` 限制，无 `allow-same-origin` 权限）。

12.1 Bridge SDK 的 Proxy 代理模式

在缺少 `allow-same-origin` 权限的沙箱中，`window.parent` 表现为跨域的 `WindowProxy`。传统通过 `window.parent.postMessage` 通信容易静默失败。针对此问题，平台提供了 Bridge SDK：

- SDK 利用 `Object.defineProperty` 与 JavaScript `Proxy`，覆盖劫持并代理沙箱内课件的 `window.parent` 和 `window.top` 对象。
- 当课件内部调用 `.postMessage()` 时，Proxy 会自动拦截并将 `targetOrigin === 'null'` 标准化修正为 `'*'`。
- 课件 HTML 资源通过 `/runtime/:uuid/` 端点分发时，服务端将自动注入 `bridge.js` 脚本。

12.2 课件上下文与 LMS API

Bridge SDK 在初始化时，会自动将以下上下文对象挂载至全局：

- `window.__LMS_STUDENT__`: 当前学生上下文数据
- `window.__LMS_COURSEWARE__`: 当前课件元数据

同时，提供全局对象 `LMS` 以调用平台能力：

- `LMS.submit(data)`: 提交交互数据或测验答案
- `LMS.saveProgress(data)`: 保存课件进度

- `LMS.finish(data)`：标记课件学习完成
- `LMS.getStudent()`：动态获取当前学生信息
- `LMS.getCourseware()`：动态获取课件信息
- `LMS.log(event, data)`：发送自定义行为日志

12.3 课堂上下文生命周期与空值契约（Nullability Contract）

在编写师生互动插件、白板挂件或沙箱浮层时，**绝不能假设课堂环境总是具备完整的班级上下文**：

1. **参数可空性现实**：
   - 教师在**单课备课试调**、**教案预览**、或者从课程库直接启动课件而**未选定上课班级**时，宿主注入的 `classId` 为 `null` 或 `undefined`。
   - ❌ **严重反模式**：直接编写硬断言：

```
// ❌ 错误示范：会导致单课备课/预览时整个组件或沙箱白屏崩溃
if (!classId) throw new Error('缺少班级上下文，无法加载数据');
```

   - ✅ **防御性降级契约**：必须支持默认防御性兜底：

```
// ✅ 规范示范：未选班级时优雅降级为单机/通用作用域
const activeClassId = classId || 'default';
```

2. **热更新与重载契约（bridge.onContextUpdate）**：
   - 教师在授课过程中可能临时切换执教班级或跳转课节，宿主不会重新销毁重建 iframe 沙箱，而是通过 `window.OpenLearnPluginBridge` 下发热更新。
   - 插件必须监听 `onContextUpdate` 以执行数据重新加载：

```
// 监听宿主课堂上下文变更（换班 / 换课）
window.OpenLearnPluginBridge.onContextUpdate((newContext) => {
  console.log('课堂上下文已变更:', newContext);
  currentClassId = newContext.classId || 'default';
  refreshData(); // 重新拉取新班级下的互动数据
});
```

附录 A：完整插件模板

```typescript
// 复制此模板开始开发你的插件
import type { PluginContext } from '@openlearn/plugin-sdk';
import {
  ICommandBusServiceToken,
  IActionRegistryServiceToken,
  IEventBusServiceToken,
  IDatabaseToken,
  IProcessServiceToken,
  IStorageServiceToken,
  IAIServiceToken,
} from '@openlearn/plugin-sdk';

export default {
  manifest: {
    id: '@you/plugin-name',
    name: '我的插件',
    version: '1.0.0',
    main: 'index.js',
    description: '插件描述',
    author: '作者名',
    engines: { openlearn: '>=0.2.5' },
    requires: [
      '@openlearn/core:ICommandBusService@^1.0.0',
      '@openlearn/core:IActionRegistryService@^1.0.0',
      '@openlearn/core:IEventBusService@^1.0.0',
      '@openlearn/core:IDatabase@^1.0.0',
    ],
    capabilitiesProposed: ['lesson:read'],
  },

  activate: async (ctx: PluginContext) => {
    const commandBus = ctx.services.commandBus;
    const actionRegistry = ctx.services.actionRegistry;
    const eventBus = ctx.services.eventBus;
    const db = await ctx.resolve(IDatabaseToken);

    // TODO: 注册 Actions 和 Handlers

    ctx.log.info('Plugin activated');
  },

  deactivate: async () => {
    console.log('插件已停用');
  },
};
```

附录 B：现有内置插件参考

插件	文件	命令示例
课堂核心	`packages/plugins/builtin.ts`	`lesson.create`, `whiteboard.draw`, `whiteboard.query`
虚拟文件系统	`packages/plugins/vfs.ts`	`vfs.write_file`, `vfs.read_file`, `vfs.list_dir`
管理插件	`packages/plugins/management.ts`	`class.create`, `student.create`, `schedule.create`
AI 规划器	`packages/plugins/ai-planner.ts`	`ai.start_generation`, `ai.apply_recommendation`
作业评估	`packages/plugins/assignment-eval.ts`	`assignment.create`, `assignment.submit`, `assignment.grade`
进程管理	`packages/plugins/process.ts`	`process.spawn`, `process.kill`, `process.list`

本文档基于 OpenLearnV2 最新代码库（`main` 分支），通过 Codegraph 知识图谱分析生成。 最后更新：2026-09-25